Redes neuronales feedforward

Unidad 7. Arquitectura, funciones de pérdida, entrenamiento y regularización del perceptrón multicapa

---

Alejandro Veloz

INF396 - Introducción a la Ciencia de Datos

Objetivos

- Describir una red neuronal feedforward como una composición de transformaciones afines y activaciones, y contar sus parámetros.
- Explicar la red ReLU como una función lineal por tramos, relacionarla con la regresión sobre bases y enunciar el teorema de aproximación universal y sus límites.
- Explicar por qué la profundidad aumenta la complejidad de las funciones representables.
- Deducir la función de pérdida de la máxima verosimilitud para regresión, regresión heterocedástica y clasificación binaria y multiclase.
- Describir el descenso de gradiente estocástico, el momentum y Adam, y reconocer el papel de la tasa de aprendizaje.
- Calcular gradientes por retropropagación y justificar la inicialización de He.
- Reconocer el doble descenso en una red, y elegir y evaluar métodos de regularización.

Notación

Se sigue la notación de Prince, con n observaciones como en el resto del curso.

Objeto	Notación
Datos de entrenamiento	$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^{n}$
Red y todos sus parámetros	$\mathbf{f}[\mathbf{x}, \phi]$
Dimensiones de entrada, capas ocultas y salida	D_i, D (o D_k), D_o
Número de capas ocultas	K
Sesgos y pesos de la capa k	$\boldsymbol{\beta}_k, \boldsymbol{\Omega}_k$
Preactivaciones y activaciones	$\mathbf{f}_k, \mathbf{h}_k$
Función de activación	$\mathrm{a}[\cdot]$, en general $\mathrm{ReLU}[\cdot]$
Pérdida total y de una observación	$L[\phi] = \sum_i \ell_i$
Tasa de aprendizaje y lote	$\alpha, \mathcal{B}_t$
Coeficientes de momentum y de Adam	ρ; ρ_1, ρ_2
Parámetros de la distribución y número de clases	$\boldsymbol{\eta}, C$

Los corchetes indican argumentos de funciones. Los sesgos $\boldsymbol{\beta}_k$ no tienen relación con los coeficientes β de la regresión lineal.

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

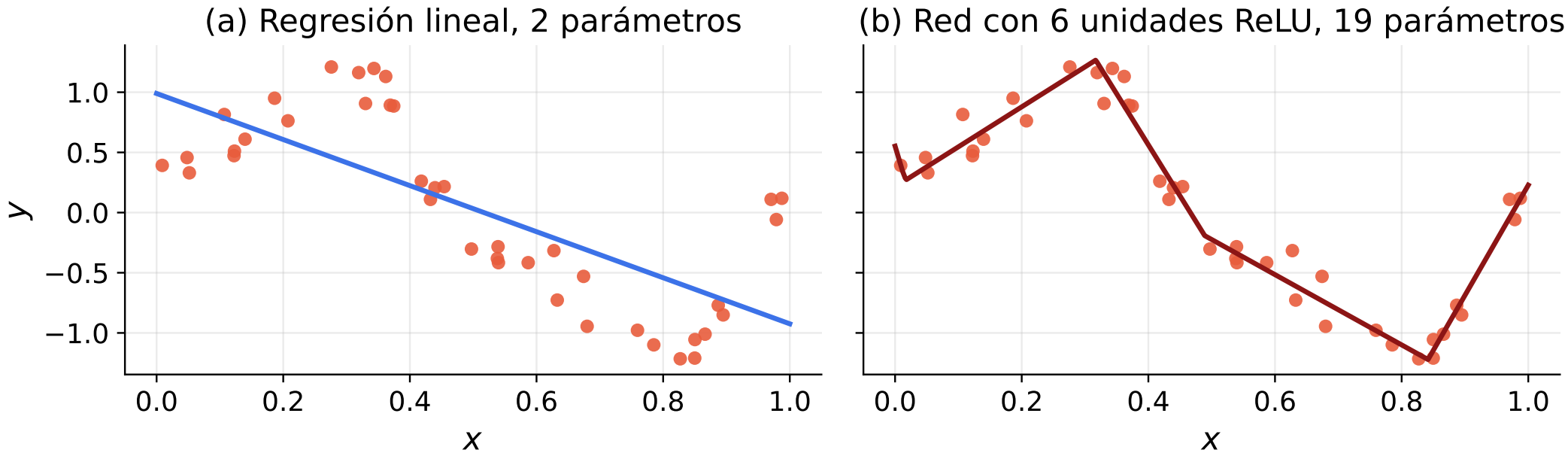
3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

El punto de partida



La regresión lineal en x describe relaciones con rectas e hiperplanos. La regresión sobre bases resuelve el problema con una base fija de funciones, por ejemplo polinomios de grado r, elegida antes de ver los datos.

Una red neuronal construye la base a partir de los datos. La red del panel (b) tiene seis unidades ocultas, 19 parámetros, y produce una función lineal por tramos cuyas juntas se ubican durante el entrenamiento.

Una red con una capa oculta

Para una entrada y una salida escalares, con tres unidades ocultas,

$$y = \mathsf{f}[x, \phi] = \phi_0 + \phi_1\, \mathsf{a}[\theta_{10} + \theta_{11}x] + \phi_2\, \mathsf{a}[\theta_{20} + \theta_{21}x] + \phi_3\, \mathsf{a}[\theta_{30} + \theta_{31}x].$$

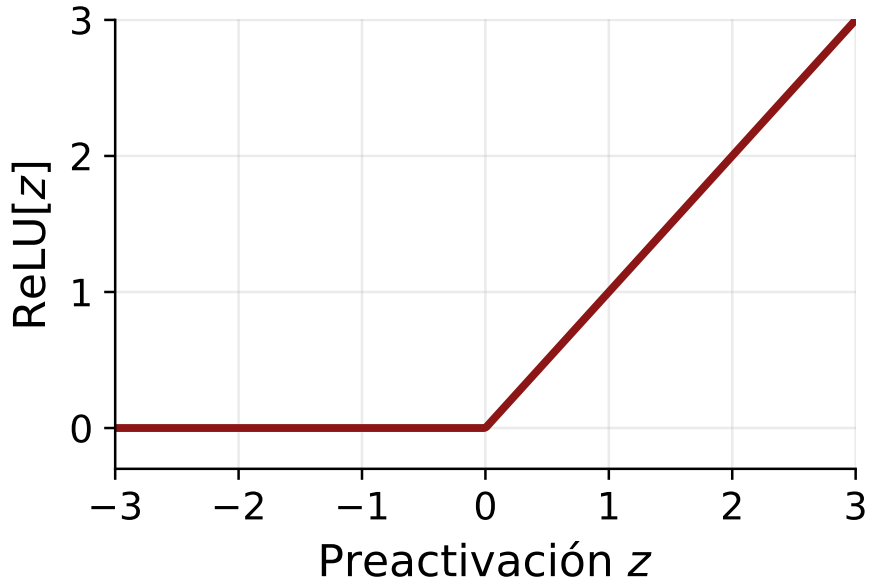
- $\mathsf{a}[\cdot]$ es la función de activación, fija y no lineal.
- $\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$ son los diez parámetros.

Con $\mathsf{a}[z] = z$ la red colapsa en una recta, $y = (\phi_0 + \sum_d \phi_d\theta_{d0}) + (\sum_d \phi_d\theta_{d1})\, x$. Toda la capacidad adicional proviene de la no linealidad de a.

La función ReLU

La activación más usada es la unidad lineal rectificada (ReLU),

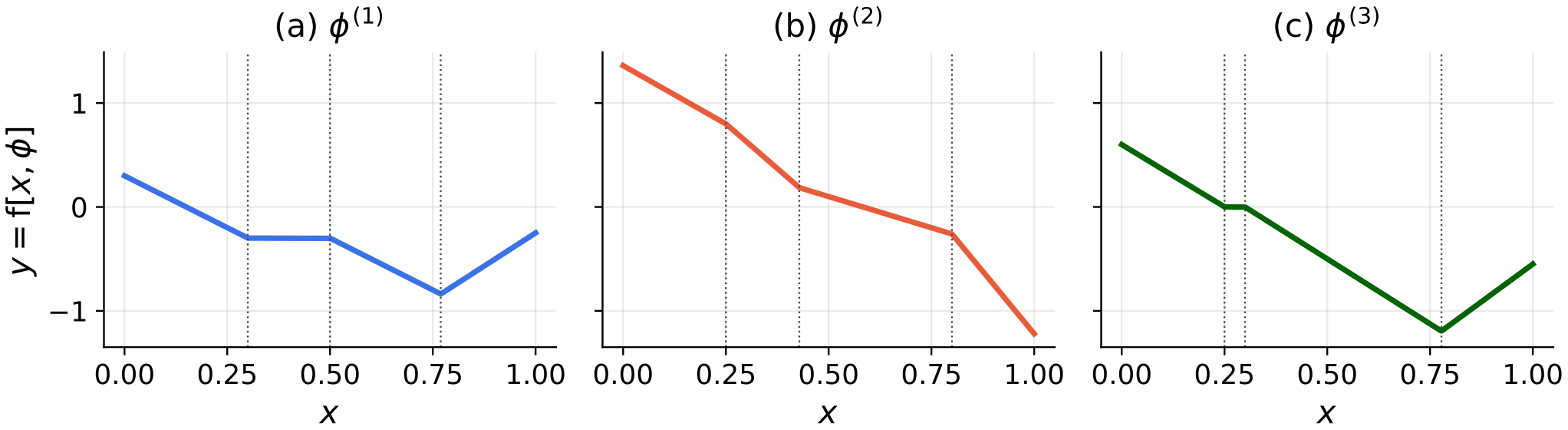
$$\text{a}[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0, \\ z & z \geq 0. \end{cases}$$



Devuelve la entrada si es positiva y cero en caso contrario. Con ella, cada término $\phi_d \, \text{ReLU}[\theta_{d0} + \theta_{d1} x]$ es una función con una sola junta en $x = -\theta_{d0}/\theta_{d1}$.

La red con D unidades ReLU es, entonces, una función lineal por tramos con hasta D juntas.

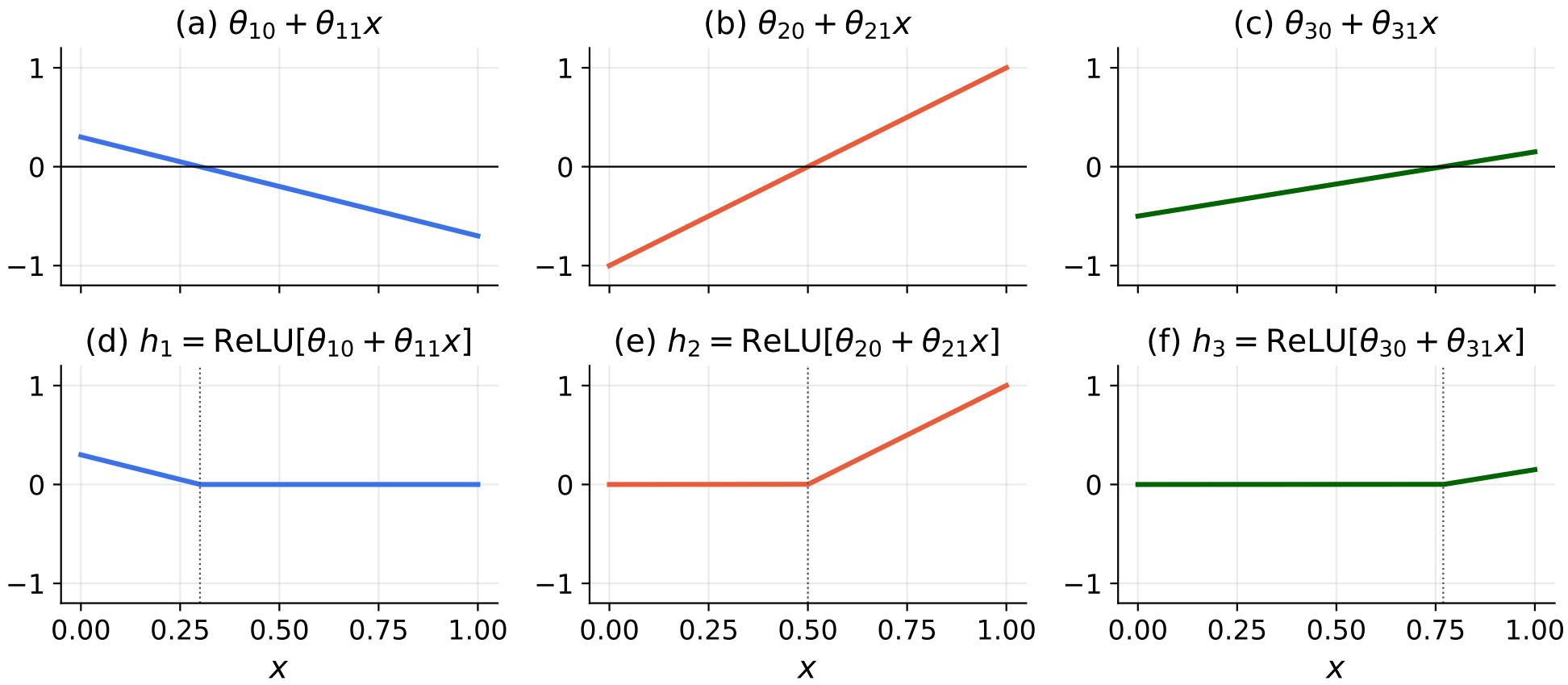
Los parámetros determinan la función



La misma ecuación con tres valores de ϕ produce tres funciones lineales por tramos. Las líneas punteadas marcan las juntas.

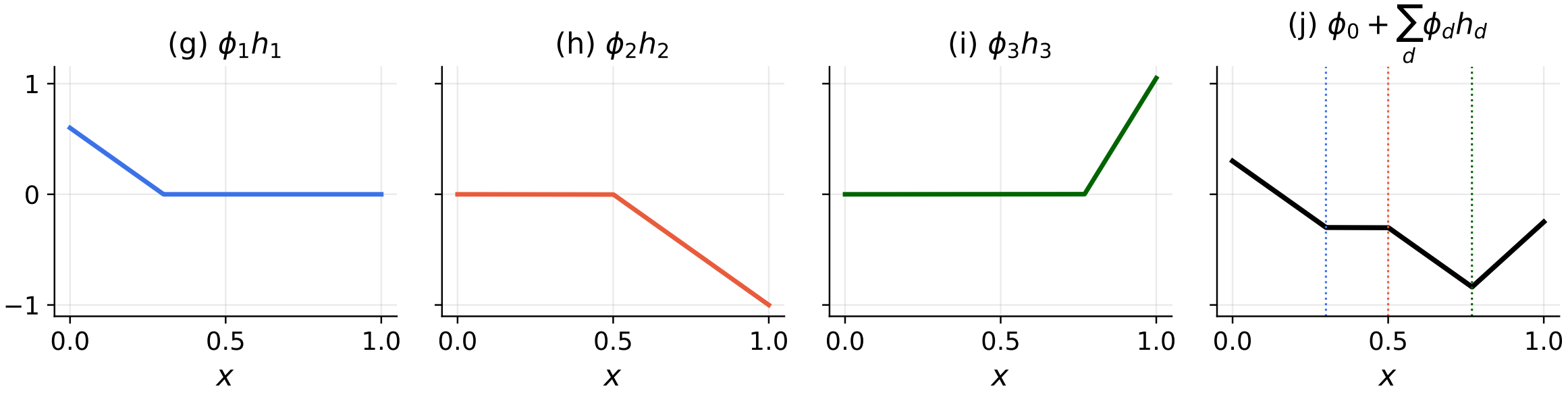
- Con los parámetros fijos, evaluar la ecuación es la **inferencia**.
- Elegir los parámetros a partir de datos $\{x_i, y_i\}_{i=1}^{n}$, minimizando una pérdida $L[\phi]$, es el **entrenamiento**.

Construcción de la salida, primer paso



Cada unidad oculta calcula una función lineal de la entrada, la **preactivación** (fila superior), y la pasa por la ReLU, que la recorta en cero y produce la **activación** h_d (fila inferior).

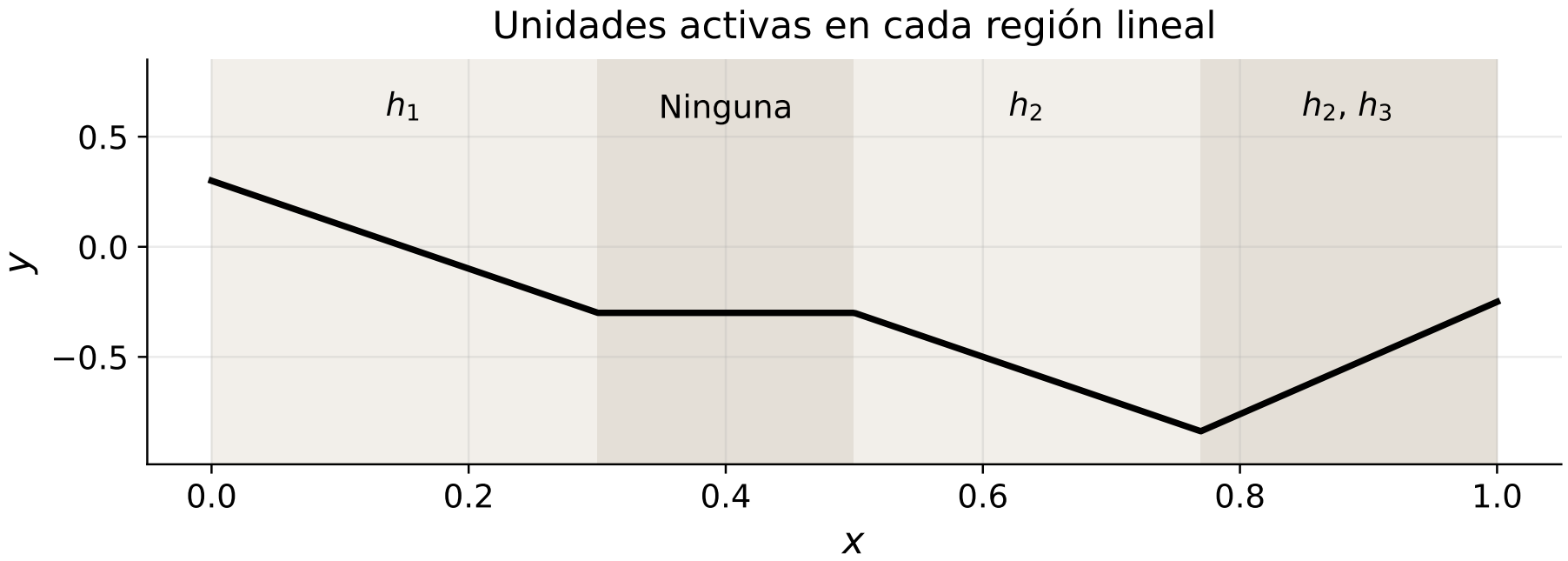
Construcción de la salida, segundo paso



Las activaciones se multiplican por los pesos de salida ϕ_d y se suman con el desplazamiento ϕ_0. Cada junta de la salida proviene de una unidad oculta, y las pendientes entre juntas son sumas de los productos $\phi_d \theta_{d1}$ de las unidades activas.

Las tres operaciones son una combinación lineal de la entrada, una función de activación y una combinación lineal de las activaciones.

Unidades activas por región



En cada región lineal está activo un conjunto distinto de unidades ocultas. Dentro de una región la red es exactamente una función lineal de x, cuya pendiente es $\sum_{d \text{ activa}} \phi_d \theta_{d1}$.

Una red ReLU es un conjunto de modelos lineales, uno por región, unidos de forma continua.

Representación gráfica

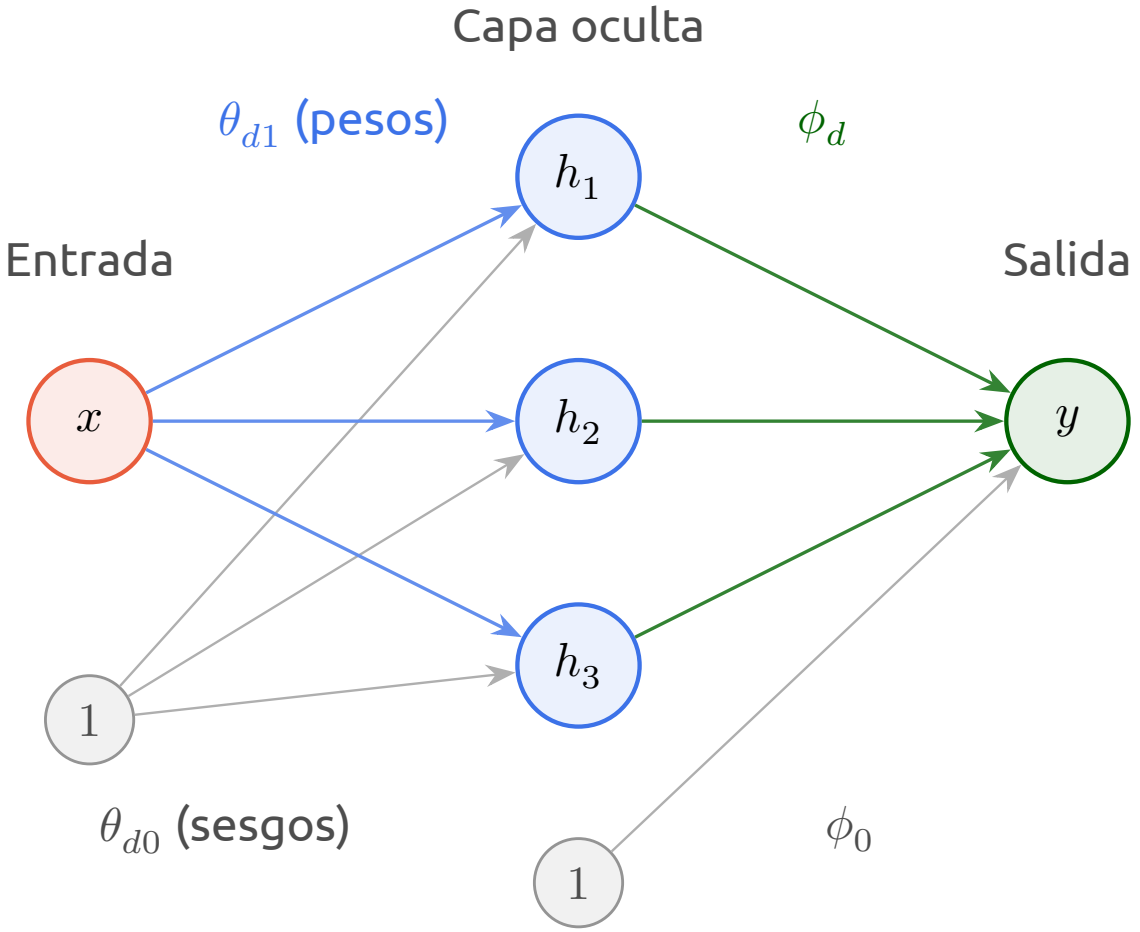
1. Combinaciones lineales de la entrada y 2. activación,

$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1} x], \quad d = 1, 2, 3.$$

3. Combinación lineal de las activaciones,

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3.$$

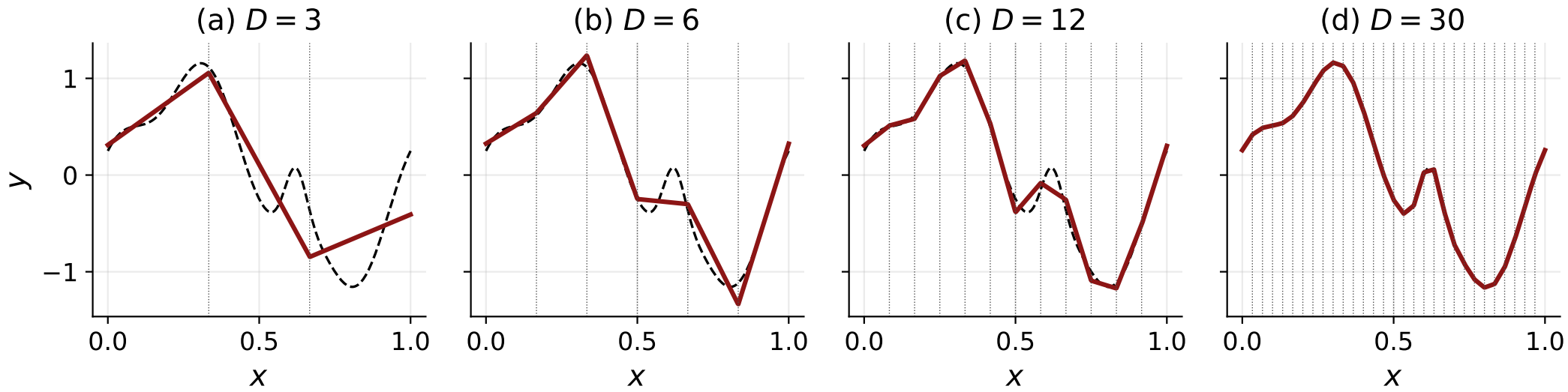
Cada flecha es un parámetro multiplicativo, un **peso**. Los nodos con un 1 representan los desplazamientos, llamados **sesgos**.



Aproximación universal

Con D unidades ocultas,

$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1}x], \qquad y = \phi_0 + \sum_{d=1}^{D} \phi_d h_d.$$



Con juntas equiespaciadas y los pesos de salida ajustados por mínimos cuadrados, el error máximo baja de $0{,}68$ con $D = 3$ a $0{,}04$ con $D = 30$.

El teorema de aproximación universal

Teorema (Cybenko, 1989; Hornik, 1991; Leshno et al., 1993). Si la activación es continua y no es un polinomio, para toda función continua g sobre un compacto $K \subset \mathbb{R}^{D_i}$ y todo $\varepsilon > 0$ existe una red con una capa oculta y D finito tal que

$$\sup_{\mathbf{x} \in K} \left| \mathsf{f}[\mathbf{x}, \phi] - g(\mathbf{x}) \right| < \varepsilon.$$

El teorema garantiza que la familia es suficientemente rica, y no dice tres cosas.

- Cuántas unidades se necesitan, que en general crece exponencialmente con D_i para funciones arbitrarias.
- Si el entrenamiento encuentra los parámetros que logran la aproximación.
- Si la función aprendida generaliza a datos nuevos.

Varias salidas

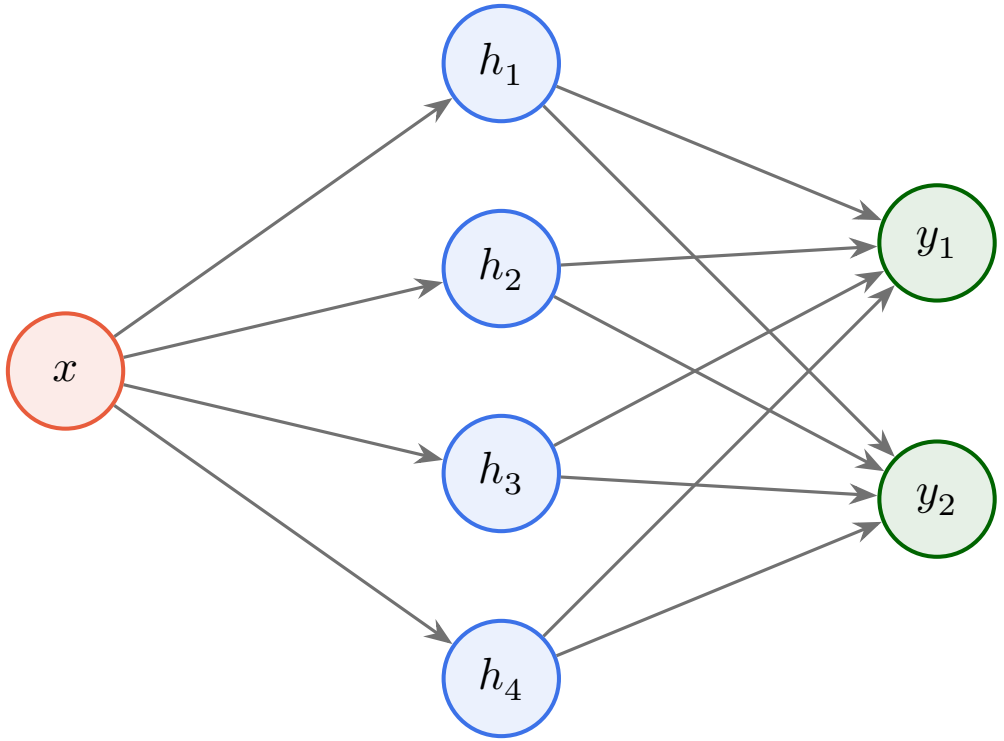
Con una entrada, cuatro unidades ocultas y dos salidas,

$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1} x], \quad d = 1, \ldots, 4,$$

$$y_1 = \phi_{10} + \sum_{d=1}^{4} \phi_{1d} h_d,$$

$$y_2 = \phi_{20} + \sum_{d=1}^{4} \phi_{2d} h_d.$$

Las dos salidas comparten las juntas, que son de las unidades ocultas, y difieren en las pendientes. Las capas ocultas son una representación común a todas las salidas.



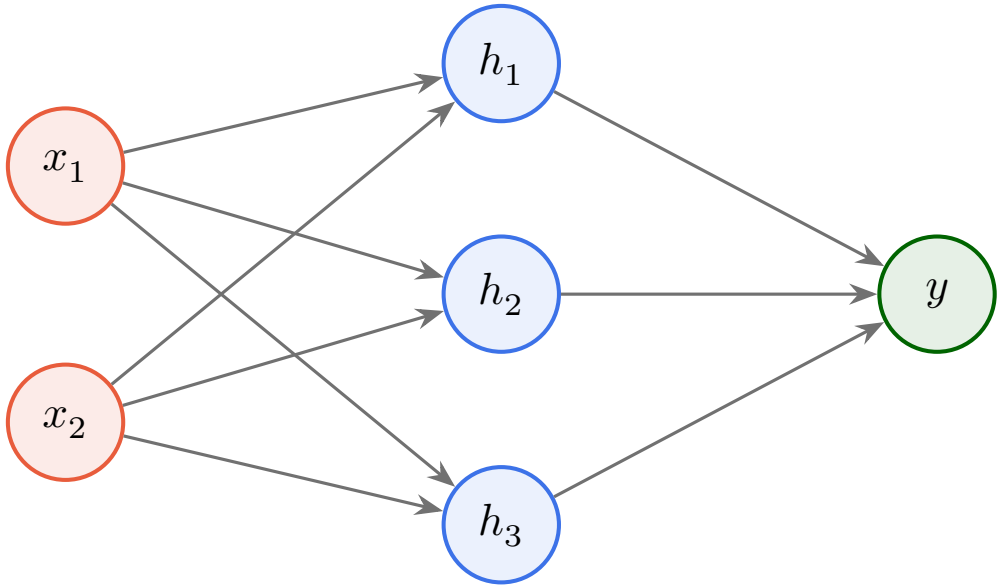
Varias entradas

Con dos entradas, tres unidades ocultas y una salida,

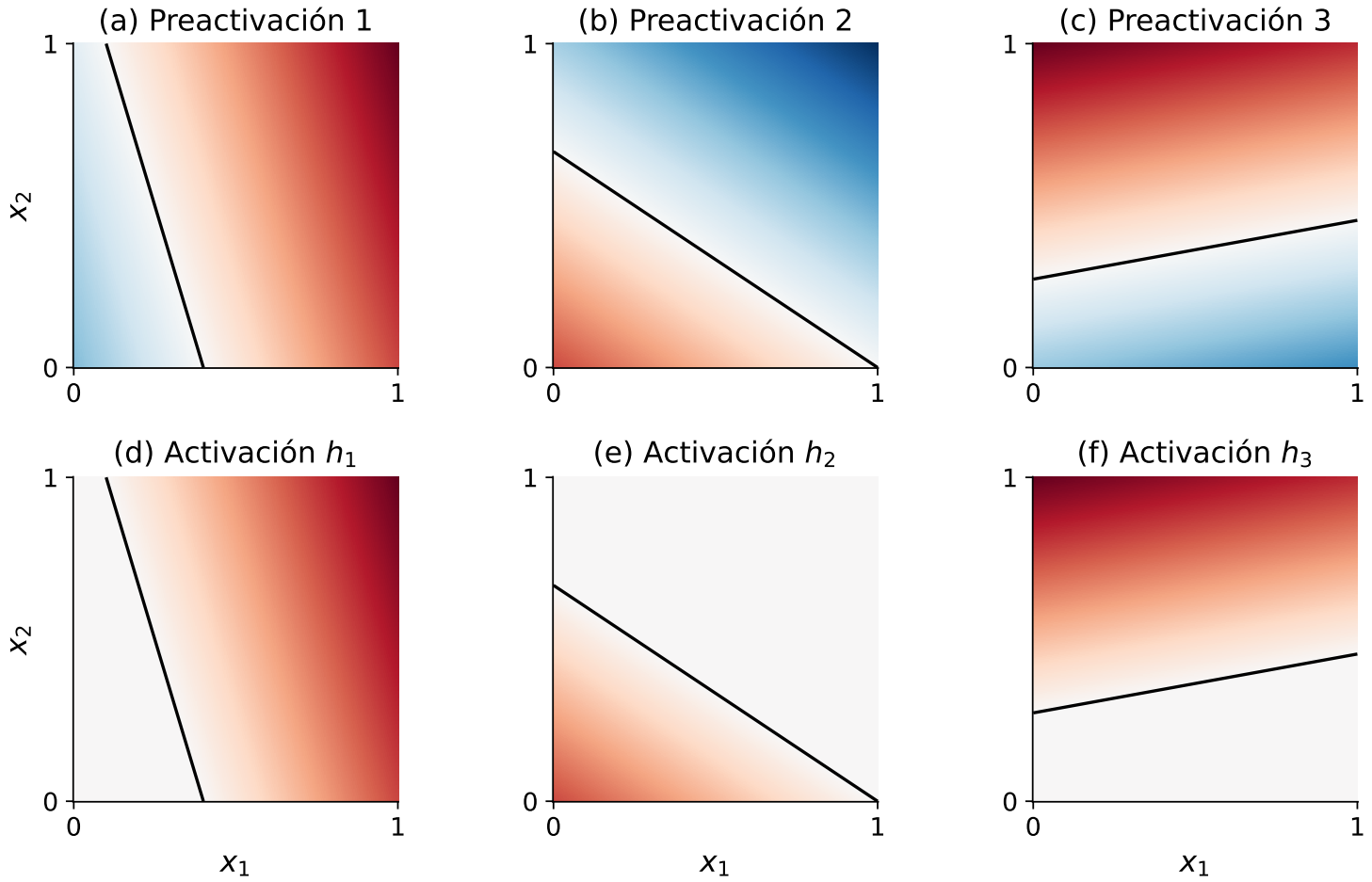
$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1}x_1 + \theta_{d2}x_2],$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3.$$

Cada preactivación es ahora un plano sobre (x_1, x_2), y la ReLU lo recorta a lo largo de la recta donde se anula. Las juntas dejan de ser puntos y pasan a ser rectas en el plano de entrada.

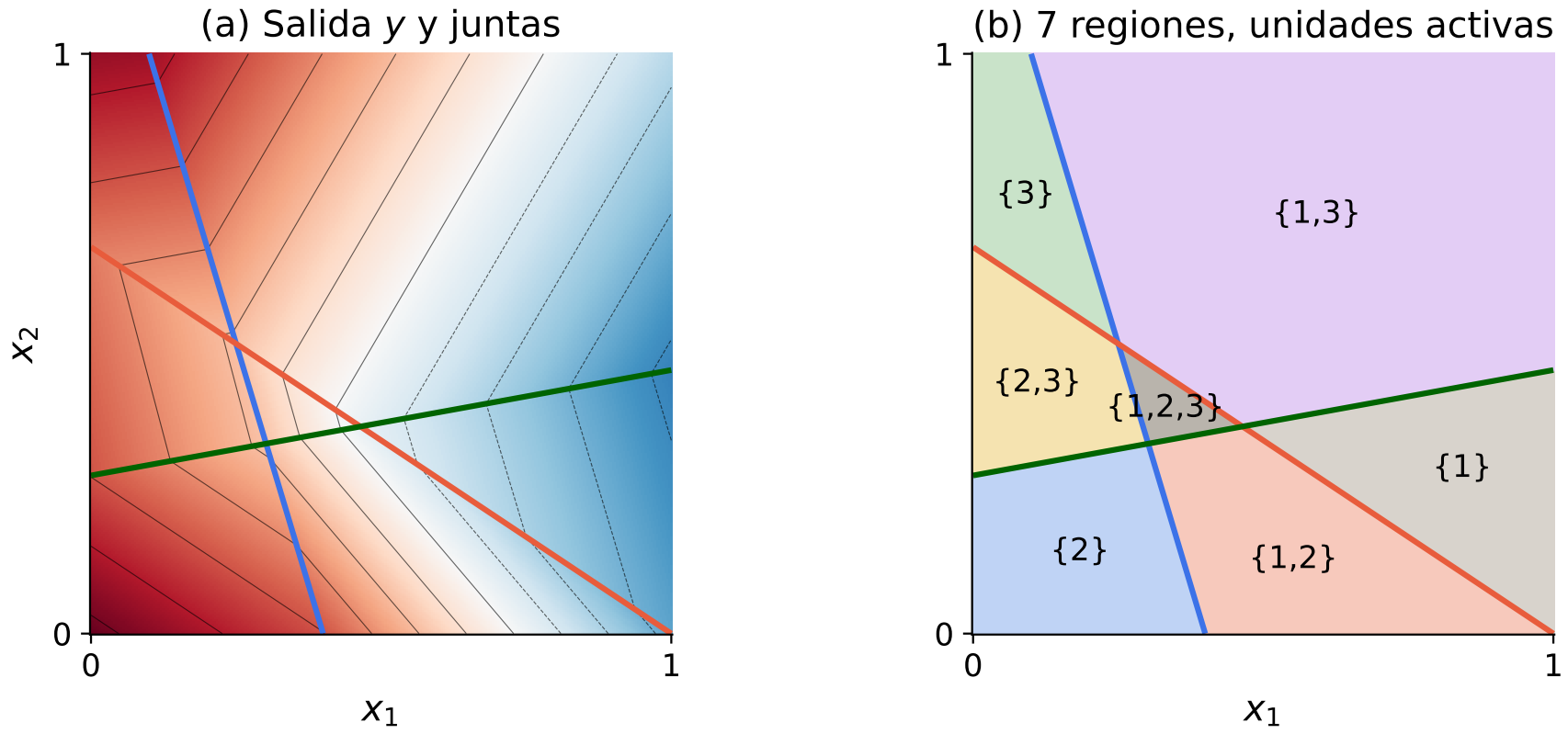


Entrada bidimensional, las unidades



Las preactivaciones son planos, y cada ReLU anula su plano a un lado de la recta negra.

Entrada bidimensional, la salida



Tres rectas dividen el cuadrado en 7 regiones convexas, cada una con su propio conjunto de unidades activas, y sobre cada región la salida es un plano.

El caso general

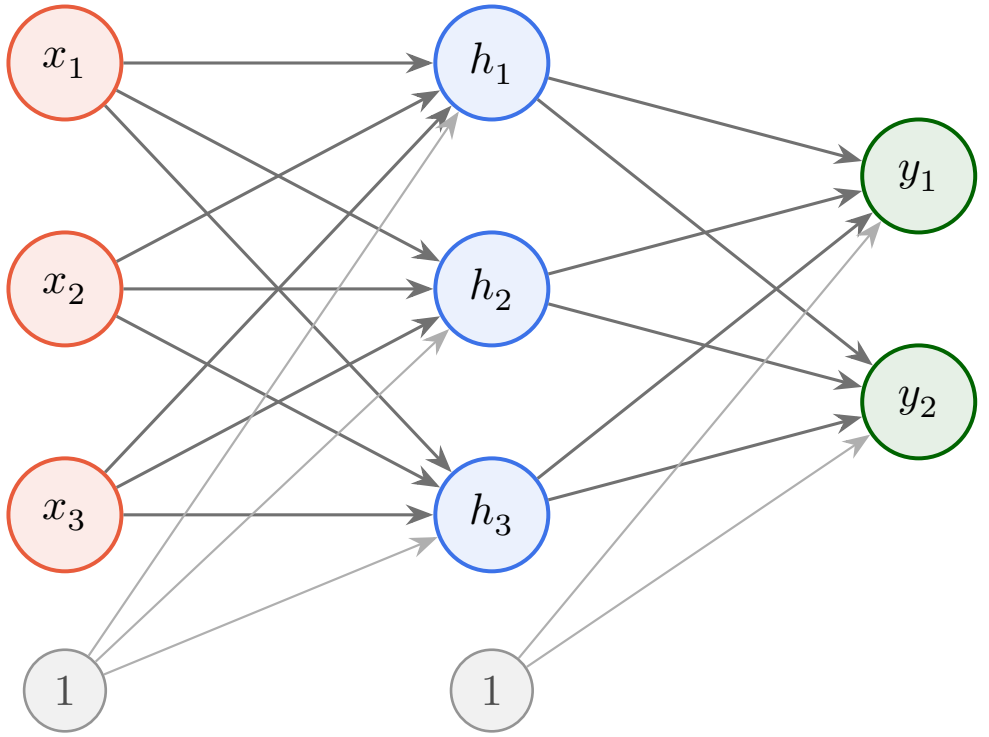
Con D_i entradas, D unidades ocultas y D_o salidas,

$$h_d = \mathsf{a}\left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i\right], \qquad y_j = \phi_{j0} + \sum_{d=1}^{D} \phi_{jd} h_d.$$

El número de parámetros es

$$D(D_i + 1) + D_o(D + 1).$$

En el diagrama, $3 \cdot 4 + 2 \cdot 4 = 20$.



Número de regiones lineales

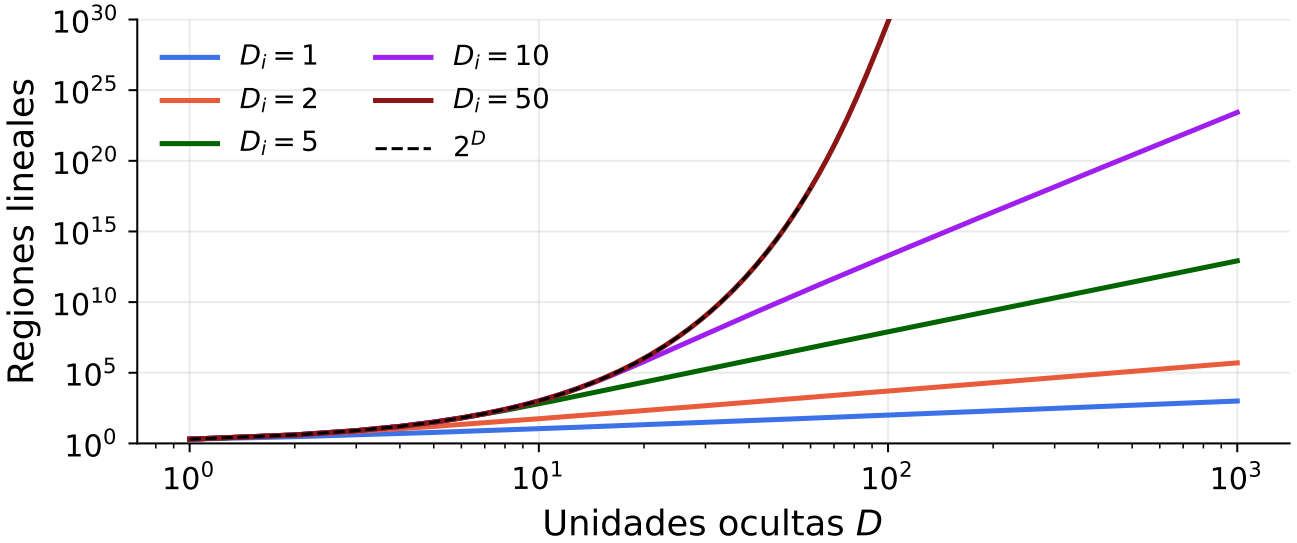
D hiperplanos en posición general dividen $\mathbb{R}^{D_i}$ en

$$\sum_{j=0}^{D_i} \binom{D}{j}$$

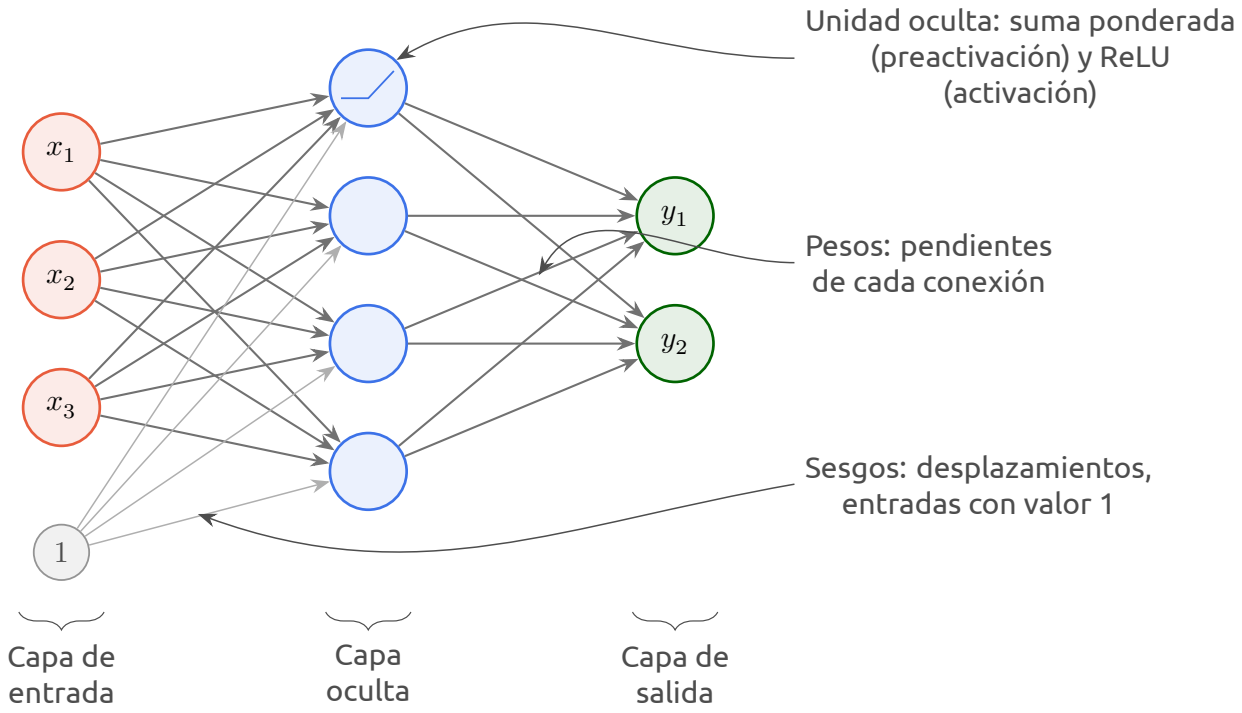
regiones (Zaslavsky, 1975).

Con $D \leq D_i$ hay 2^D regiones. Con $D > D_i$ el número queda entre 2^{D_i} y 2^D, y crece como D^{D_i}.

Con $D_i = 10$, $D = 500$ y una salida, hasta $2{,}5 \cdot 10^{20}$ regiones lineales con $6\,001$ parámetros.

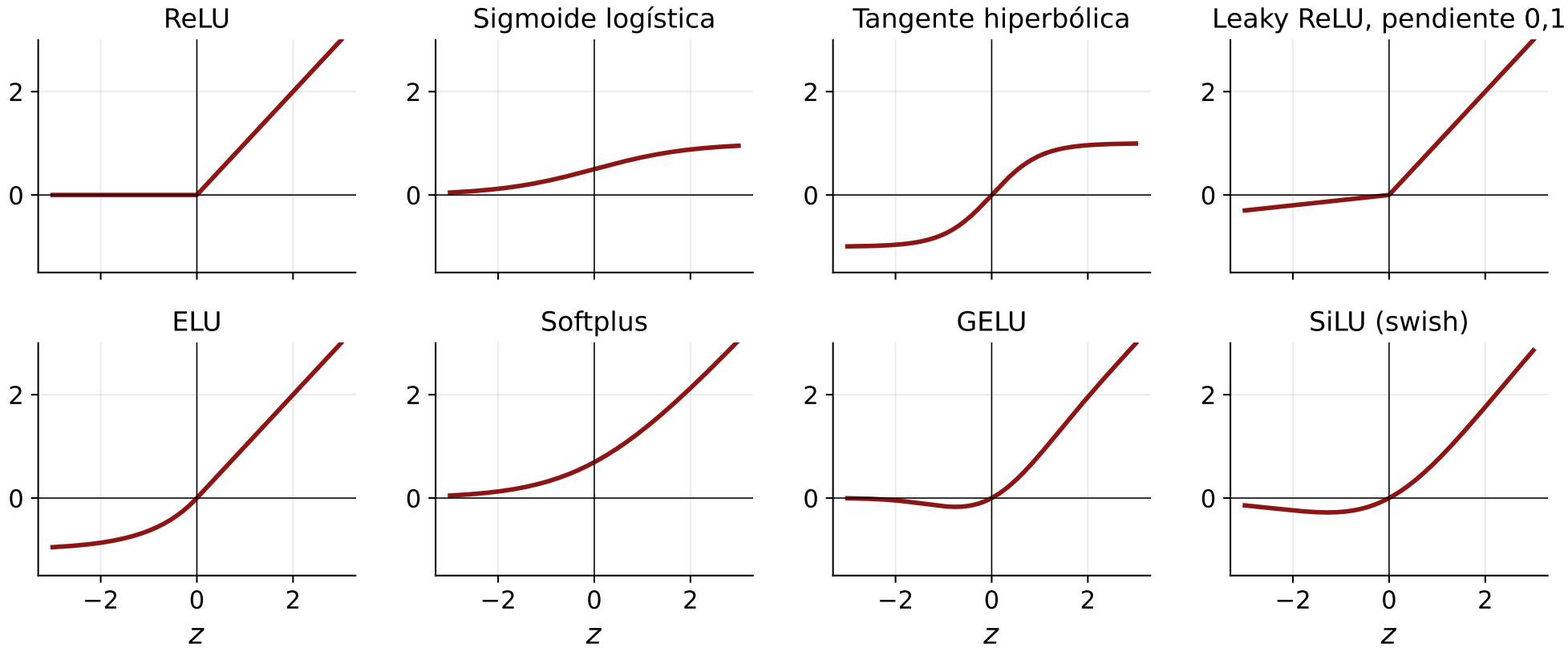


Terminología



- Pendientes, **pesos**. Desplazamientos, **sesgos**.
- Todas las unidades de una capa conectadas con todas las de la siguiente, red **completamente conectada**.
- Sin ciclos, red **feedforward** (de propagación hacia adelante).
- Valores antes de la activación, **preactivaciones**; después, **activaciones**.
- Una capa oculta, red **poco profunda**; más de una, red **profunda**.
- Una red completamente conectada y feedforward es un **perceptrón multicapa** (MLP).

Otras funciones de activación



La sigmoide y la tangente hiperbólica saturan y sus derivadas se anulan para $|z|$ grande. Las variantes de la ReLU evitan las unidades muertas con una pendiente no nula para $z < 0$. GELU (Gaussian error linear unit) y SiLU (sigmoid linear unit) son suaves y se usan en los transformers.

La red como regresión sobre una base aprendida

Con la capa oculta fija, la red poco profunda es una regresión lineal sobre la base

$$\{1,\ \mathsf{a}[\theta_{10} + \boldsymbol{\theta}_1^\top \mathbf{x}],\ \ldots,\ \mathsf{a}[\theta_{D0} + \boldsymbol{\theta}_D^\top \mathbf{x}]\},$$

y los pesos de salida tienen la solución de mínimos cuadrados.

Entrenar la red ajusta también la base, es decir, la posición y la orientación de cada junta.

	Regresión sobre bases	**Red neuronal**
Base	Fija, polinomios o splines	Aprendida de los datos
Parámetros	Lineales en el modelo	No lineales en θ
Ajuste	Solución cerrada	Optimización iterativa
Capacidad	Grado r	Ancho D y profundidad K
Regularización	Ridge, lasso	L2, parada temprana, dropout y otras

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

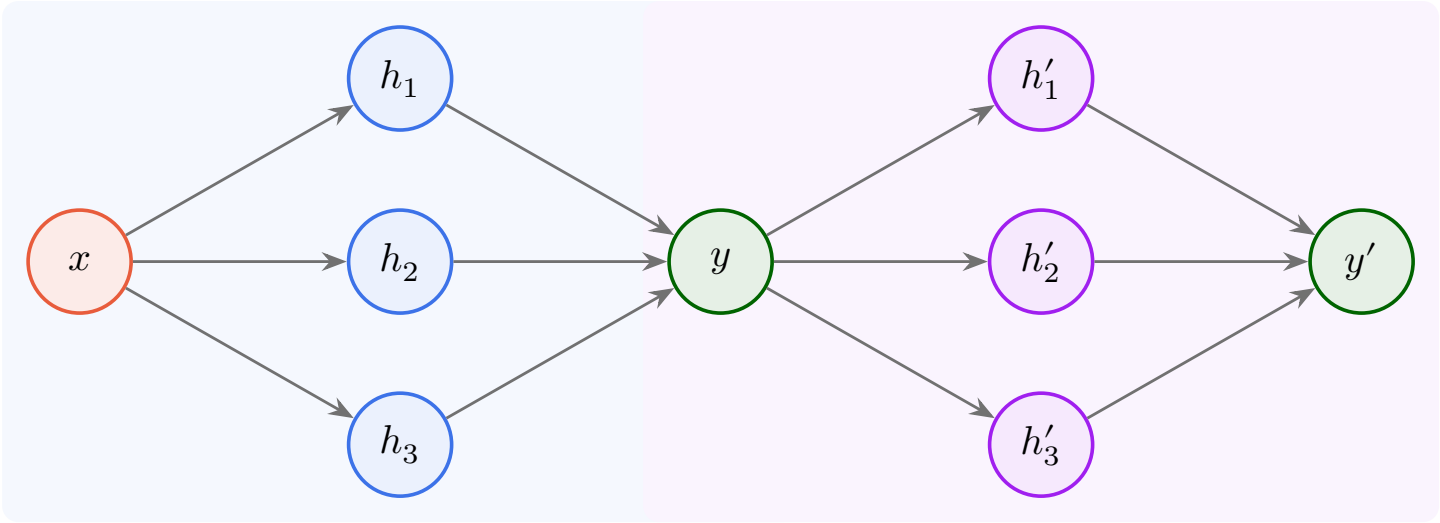
3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

Componer dos redes



Red 1 Red 2

Red 1,

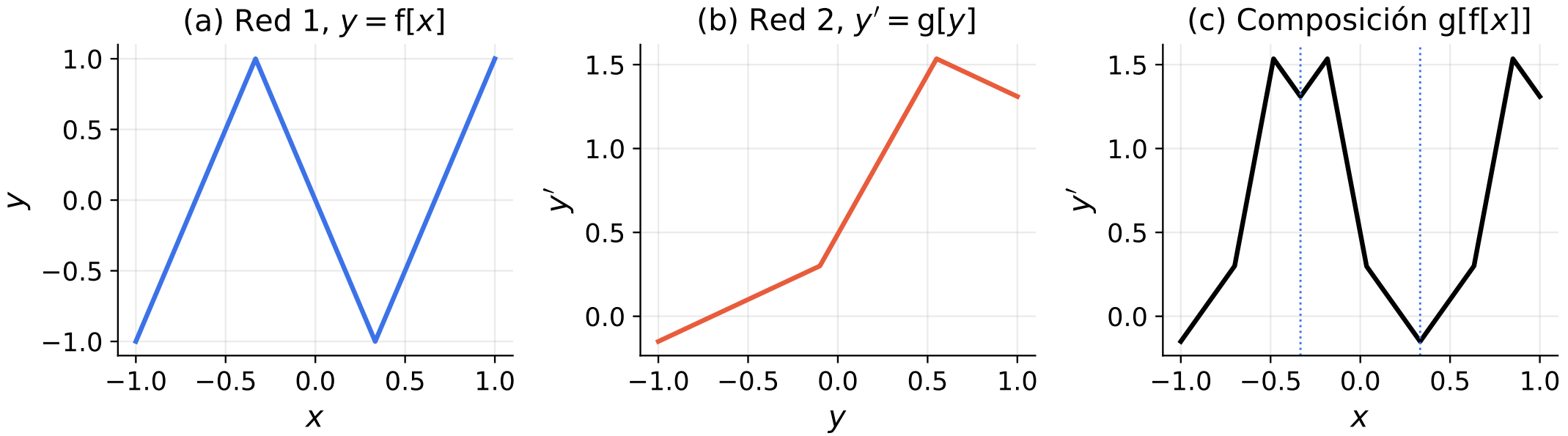
$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1}x], \quad y = \phi_0 + \textstyle\sum_d \phi_d h_d.$$

Red 2, aplicada a la salida de la primera,

$$h'_d = \mathsf{a}[\theta'_{d0} + \theta'_{d1}y], \quad y' = \phi'_0 + \textstyle\sum_d \phi'_d h'_d.$$

Una red profunda se obtiene, en su forma más simple, usando la salida de una red como entrada de otra.

La composición pliega la entrada



La red 1 recorre tres veces el intervalo $[-1, 1]$, subiendo, bajando y volviendo a subir. La red 2 tiene tres regiones lineales sobre ese intervalo. La composición repite la red 2 tres veces, la del medio invertida, y tiene 9 regiones lineales.

Con 20 parámetros la composición tiene 9 regiones; una red poco profunda con 6 unidades, 19 parámetros, tiene como máximo 7.

De dos redes a una red con dos capas ocultas

Sustituyendo y en la segunda red,

$$h'_1 = \mathsf{a}[\theta'_{10} + \theta'_{11}y] = \mathsf{a}[\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1 h_1 + \theta'_{11}\phi_2 h_2 + \theta'_{11}\phi_3 h_3],$$

y lo mismo para h'_2 y h'_3.

Con $\psi_{10} = \theta'_{10} + \theta'_{11}\phi_0$ y $\psi_{1d} = \theta'_{11}\phi_d$,

$$h'_k = \mathsf{a}[\psi_{k0} + \psi_{k1}h_1 + \psi_{k2}h_2 + \psi_{k3}h_3], \qquad k = 1, 2, 3.$$

La composición es un caso particular de una red con dos capas ocultas, en el que la matriz $\Psi = [\psi_{kd}]$ tiene rango uno. Dejar libres los nueve ψ_{kd} amplía la familia.

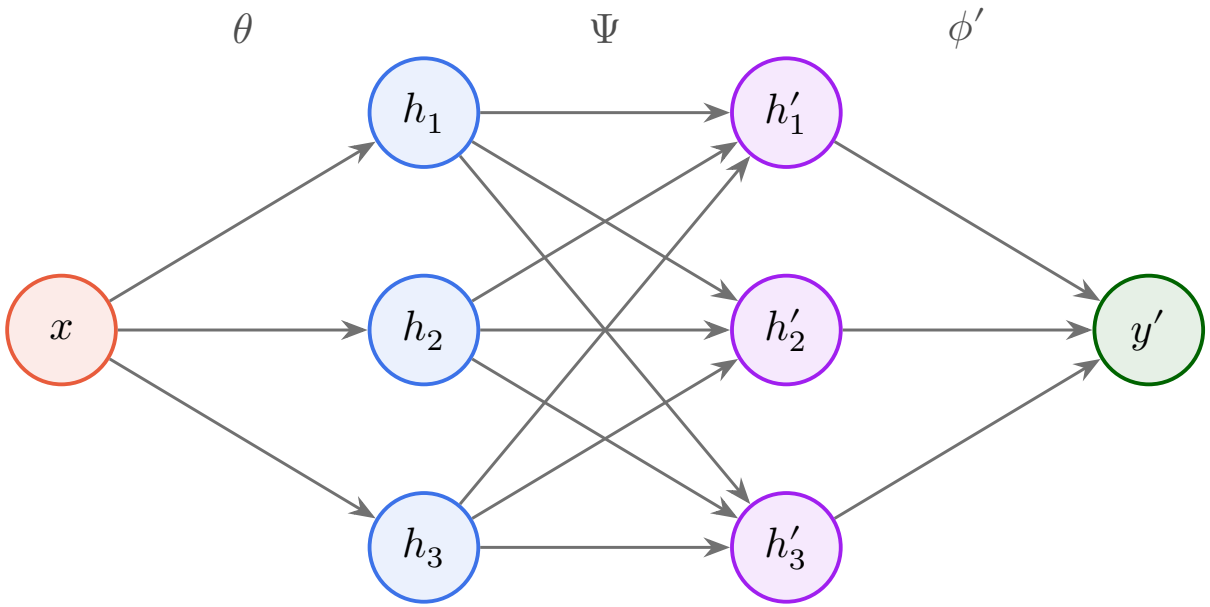
Una red con dos capas ocultas

$$h_d = \mathsf{a}[\theta_{d0} + \theta_{d1} x],$$

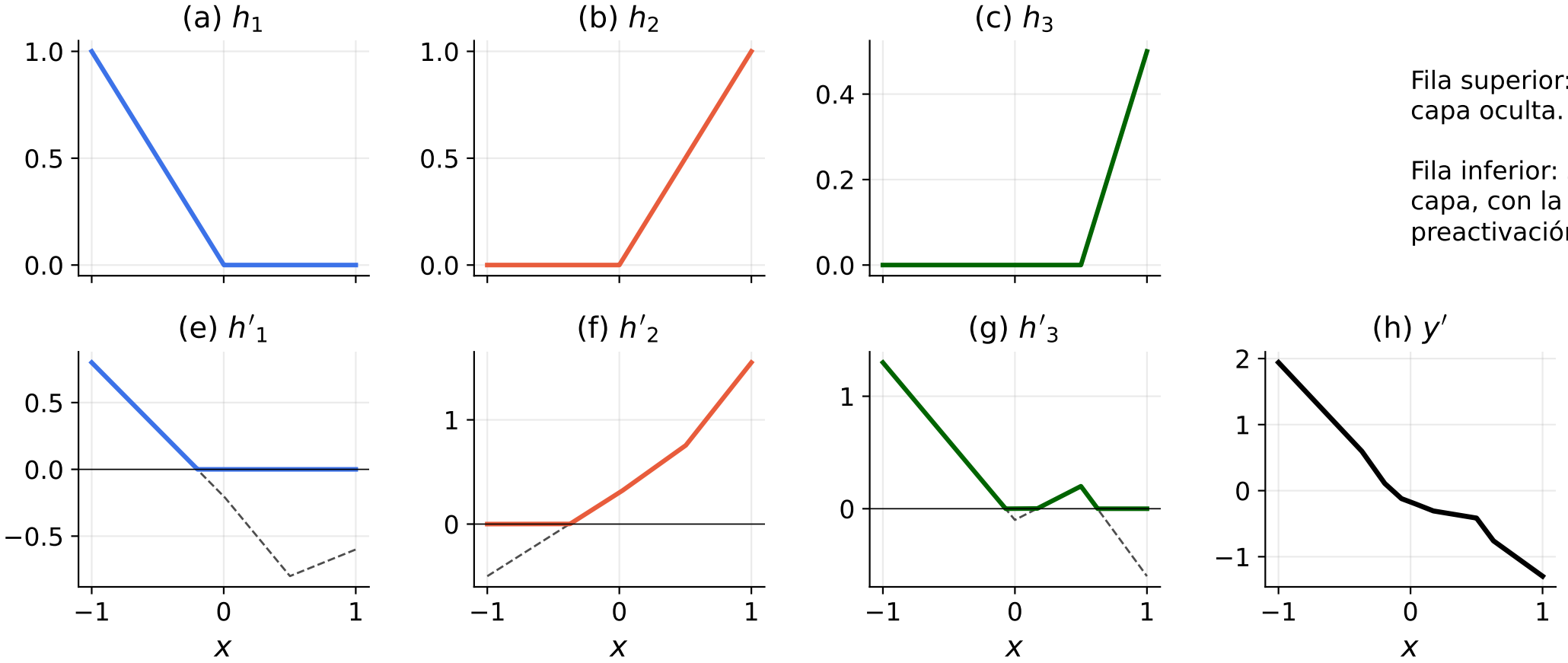
$$h'_k = \mathsf{a}\left[\psi_{k0} + \sum_{d=1}^{3} \psi_{kd} h_d\right],$$

$$y' = \phi'_0 + \sum_{k=1}^{3} \phi'_k h'_k.$$

Cada unidad de la segunda capa recibe una función lineal por tramos de x, la recorta en cero y la entrega a la salida.



Dos capas, unidad por unidad



Fila superior: primera capa oculta.

Fila inferior: segunda capa, con la preactivación en gris.

La segunda capa combina funciones lineales por tramos (en gris) y las recorta en cero, lo que agrega juntas nuevas en los puntos donde cada combinación cruza el cero.

Notación matricial

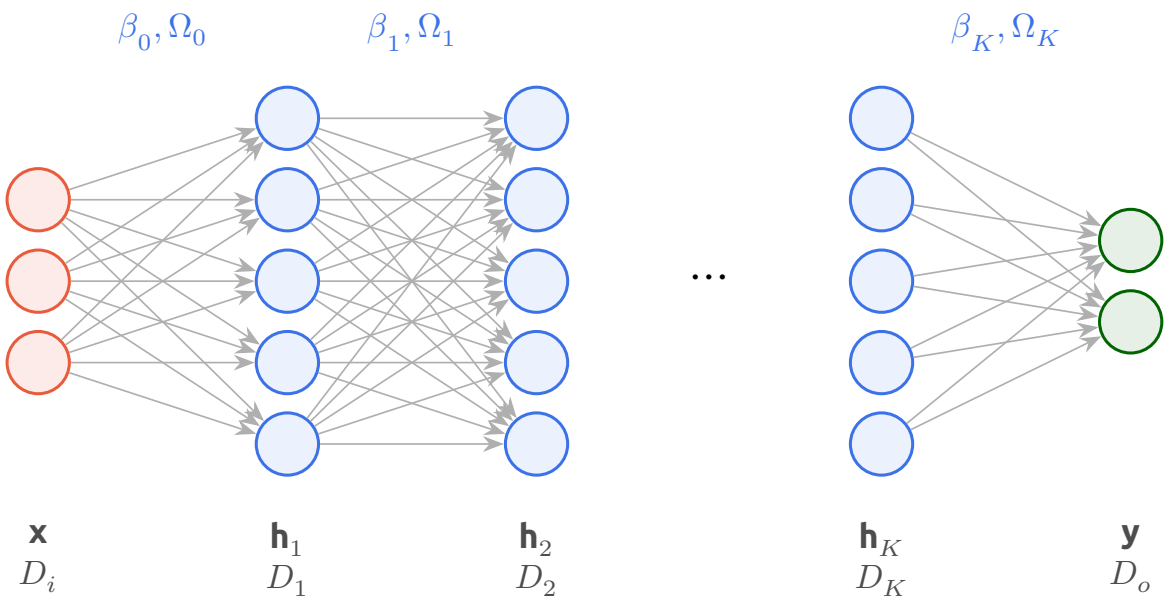
Con vectores y matrices, aplicando $\mathrm{a}[\cdot]$ elemento a elemento,

$$\begin{aligned}\mathrm{h} &= \mathrm{a}[\boldsymbol{\theta}_0 + \boldsymbol{\theta} x],\\ \mathrm{h}' &= \mathrm{a}[\boldsymbol{\psi}_0 + \boldsymbol{\Psi}\mathrm{h}],\\ y' &= \phi_0' + \boldsymbol{\phi}'^{\top}\mathrm{h}'.\end{aligned}$$

En general, con K capas ocultas, sesgos $\boldsymbol{\beta}_k$ y matrices de pesos $\boldsymbol{\Omega}_k$,

$$\begin{aligned}\mathrm{h}_1 &= \mathrm{a}[\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0\mathrm{x}],\\ \mathrm{h}_{k+1} &= \mathrm{a}[\boldsymbol{\beta}_k + \boldsymbol{\Omega}_k\mathrm{h}_k], \quad k = 1, \ldots, K-1,\\ \mathrm{y} &= \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K\mathrm{h}_K.\end{aligned}$$

El perceptrón multicapa



El número de parámetros es

$$\underbrace{D_1(D_i+1)}_{\text{entrada}} + \sum_{k=1}^{K-1} \underbrace{D_{k+1}(D_k+1)}_{\text{capas ocultas}} + \underbrace{D_o(D_K+1)}_{\text{salida}}.$$

La profundidad K y los anchos D_k son **hiperparámetros**, que no se eligen por el entrenamiento.

La red como composición de funciones

Escrita como una sola expresión,

$$\mathrm{y} = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K\, \mathrm{a}\Big[\boldsymbol{\beta}_{K-1} + \boldsymbol{\Omega}_{K-1}\, \mathrm{a}\big[\cdots\, \mathrm{a}[\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathrm{x}]\cdots\big]\Big],$$

es decir $\mathrm{y} = (\mathsf{f}_K \circ \mathsf{f}_{K-1} \circ \cdots \circ \mathsf{f}_0)(\mathrm{x})$, con f_k una transformación afín seguida de la activación.

La ReLU es positivamente homogénea, $\mathrm{ReLU}[sz] = s\,\mathrm{ReLU}[z]$ para $s > 0$. Multiplicar los pesos de una capa por s y dividir los de la siguiente por s no cambia la función,

$$\boldsymbol{\Omega}_{k+1}\,\mathrm{ReLU}[\boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathrm{h}] = \tfrac{1}{s}\boldsymbol{\Omega}_{k+1}\,\mathrm{ReLU}[s\boldsymbol{\beta}_k + s\boldsymbol{\Omega}_k \mathrm{h}].$$

Los parámetros de una red no son identificables, a diferencia de los coeficientes de una regresión lineal.

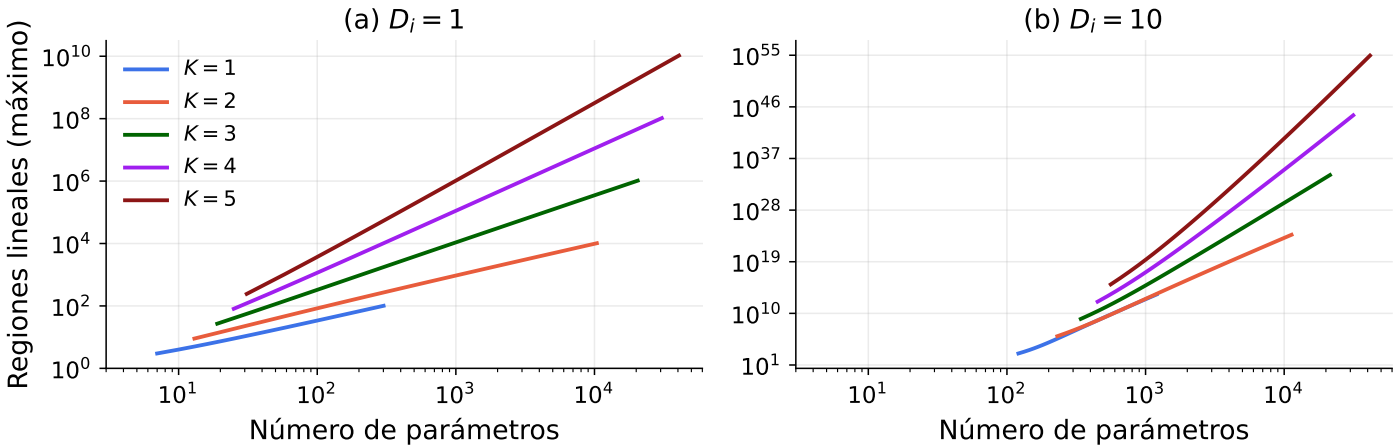
Profundidad frente a ancho

El máximo de regiones de una red con K capas de $D \geq D_i$ unidades es al menos

$$\left(\frac{D}{D_i}+1\right)^{D_i(K-1)} \sum_{j=0}^{D_i} \binom{D}{j}$$

(Montúfar et al., 2014).

Para el mismo número de parámetros, cada capa adicional multiplica el máximo de regiones.



Razones para usar redes profundas

- **Eficiencia de representación.** Existen funciones que una red de profundidad K representa con un número polinomial de unidades y que una red menos profunda sólo aproxima con un número exponencial (Telgarsky, 2016; Eldan y Shamir, 2016).
- **Composicionalidad.** Muchos problemas tienen estructura jerárquica, como los bordes que forman partes que forman objetos en una imagen, y la composición de capas la refleja.
- **Entrenamiento.** Empíricamente, las redes profundas y moderadamente anchas se entrenan y generalizan mejor que las poco profundas y muy anchas con el mismo número de parámetros.

Ninguno de estos argumentos es un teorema sobre los datos reales. El número de capas y su ancho se eligen por validación, como el grado de un polinomio en la regresión sobre bases.

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

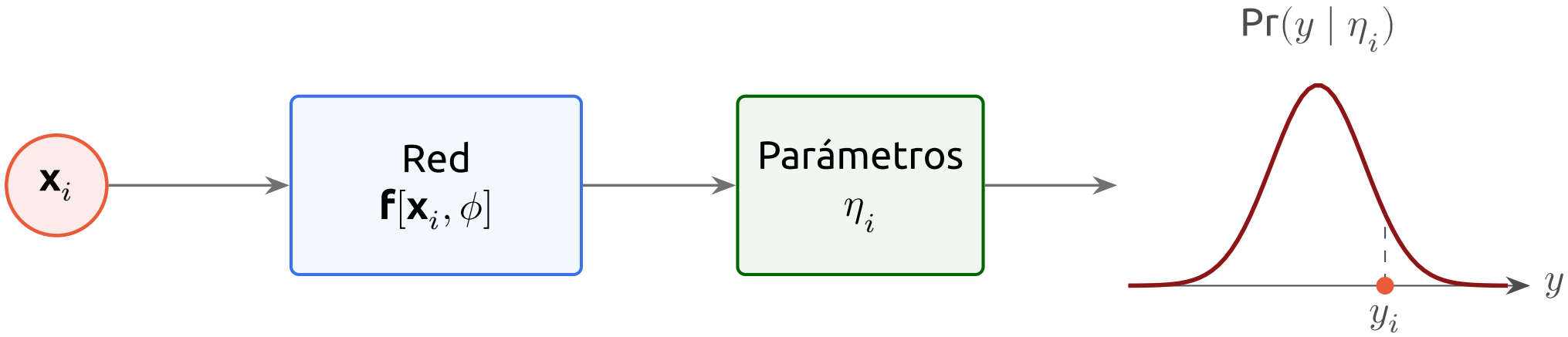
La función de pérdida

Los parámetros ϕ determinan la función $\mathrm{f}[\mathrm{x}, \phi]$. Entrenar es elegir ϕ de modo que la función describa bien los datos $\{\mathrm{x}_i, \mathrm{y}_i\}_{i=1}^n$, y la **función de pérdida** $L[\phi]$ resume en un número cuánto se aparta la predicción $\mathrm{f}[\mathrm{x}_i, \phi]$ de y_i,

$$\hat{\phi} = \underset{\phi}{\text{argmin}}\ L[\phi].$$

Hasta ahora la pérdida se eligió caso a caso, la cuadrática para regresión y la entropía cruzada para la regresión logística. Esta parte la deduce de un principio, la máxima verosimilitud, con lo que una misma receta produce la pérdida adecuada para cualquier tipo de respuesta.

La red predice una distribución



Pérdida: $-\log \Pr(y_i \mid \eta_i)$

Se elige una distribución $\Pr(\mathrm{y} \mid \boldsymbol{\eta})$ sobre el dominio de la respuesta, y la red calcula sus parámetros para cada entrada, $\boldsymbol{\eta}_i = \mathrm{f}[\mathrm{x}_i, \phi]$. La pérdida de una observación es $-\log \Pr(\mathrm{y}_i \mid \boldsymbol{\eta}_i)$, pequeña cuando la distribución predicha da probabilidad alta al valor observado.

El criterio de máxima verosimilitud

Bajo dos supuestos,

1. todas las observaciones siguen la misma familia de distribuciones, y
2. las observaciones son independientes dado x,

la probabilidad conjunta de las respuestas es un producto, y se elige

$$\hat{\phi} = \underset{\phi}{\text{argmax}} \left[\prod_{i=1}^{n} \text{Pr}\left(\mathrm{y}_i \mid \mathrm{f}[\mathrm{x}_i, \phi]\right) \right].$$

El producto de muchas probabilidades pequeñas se pierde en la aritmética de punto flotante. El logaritmo es creciente, no mueve el máximo y convierte el producto en suma. Por convención se minimiza el negativo, la **log verosimilitud negativa**,

$$L[\phi] = -\sum_{i=1}^{n} \log \text{Pr}\left(\mathrm{y}_i \mid \mathrm{f}[\mathrm{x}_i, \phi]\right).$$

La receta

1. Elegir una distribución $\Pr(\mathrm{y} \mid \boldsymbol{\eta})$ definida sobre el dominio de las respuestas.
2. Hacer que la red prediga uno o más de sus parámetros, $\boldsymbol{\eta} = \mathrm{f}[\mathrm{x}, \phi]$, con las transformaciones necesarias para respetar sus restricciones.
3. **Entrenar** minimizando la log verosimilitud negativa sobre los datos,

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^{n} \log \Pr\left(\mathrm{y}_i \mid \mathrm{f}[\mathrm{x}_i, \phi]\right) \right].$$

4. **Inferir**, para una entrada nueva x, la distribución completa $\Pr(\mathrm{y} \mid \mathrm{f}[\mathrm{x}, \hat{\phi}])$ o un resumen, como su moda,

$$\hat{\mathrm{y}} = \underset{\mathrm{y}}{\operatorname{argmax}} \; \Pr\left(\mathrm{y} \mid \mathrm{f}[\mathrm{x}, \hat{\phi}]\right).$$

Regresión, la distribución normal

Con $y \in \mathbb{R}$ y $\text{Pr}(y \mid \mu, \sigma^2) = N(\mu, \sigma^2)$, la red predice la media, $\mu = \text{f}[\text{x}, \phi]$, y

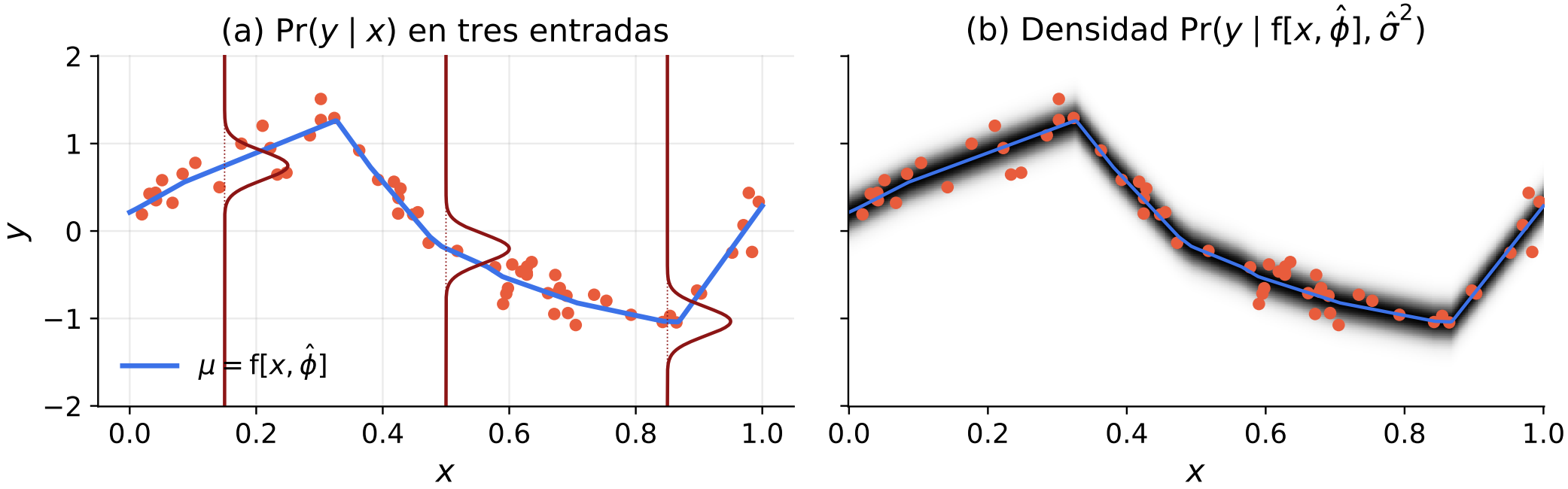
$$\begin{aligned} L[\phi] &= -\sum_{i=1}^{n} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \text{f}[\text{x}_i, \phi])^2}{2\sigma^2}\right)\right] \\ &= \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \text{f}[\text{x}_i, \phi])^2. \end{aligned}$$

El primer término y el factor $1/2\sigma^2$ no dependen de ϕ, de modo que

$$\hat{\phi} = \underset{\phi}{\text{argmin}} \sum_{i=1}^{n}(y_i - \text{f}[\text{x}_i, \phi])^2.$$

Mínimos cuadrados es la máxima verosimilitud con errores normales independientes de varianza constante.

La red predice la media de una normal



Red con dos capas ocultas de 32 unidades entrenada con pérdida cuadrática sobre 60 observaciones. Para cada x la red entrega una distribución normal centrada en $\text{f}[x, \hat{\phi}]$, con una varianza común estimada por el error cuadrático medio de entrenamiento.

Estimar la varianza

La varianza σ^2 puede tratarse como un parámetro más y estimarse junto con ϕ,

$$\hat{\phi}, \hat{\sigma}^2 = \underset{\phi, \sigma^2}{\text{argmin}} \left[\frac{n}{2} \log \sigma^2 + \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - \mathsf{f}[\mathbf{x}_i, \phi] \right)^2 \right].$$

La media no depende de σ^2, y para ϕ fijo el óptimo es $\hat{\sigma}^2 = \frac{1}{n} \sum_i (y_i - \mathsf{f}[\mathbf{x}_i, \hat{\phi}])^2$.

Esta varianza es la misma para todas las entradas. Si el ruido depende de $\mathbf{x}$, el modelo es **heterocedástico** y la red tiene que predecir también la varianza.

Regresión heterocedástica

La red tiene dos salidas, la media y el logaritmo de la varianza,

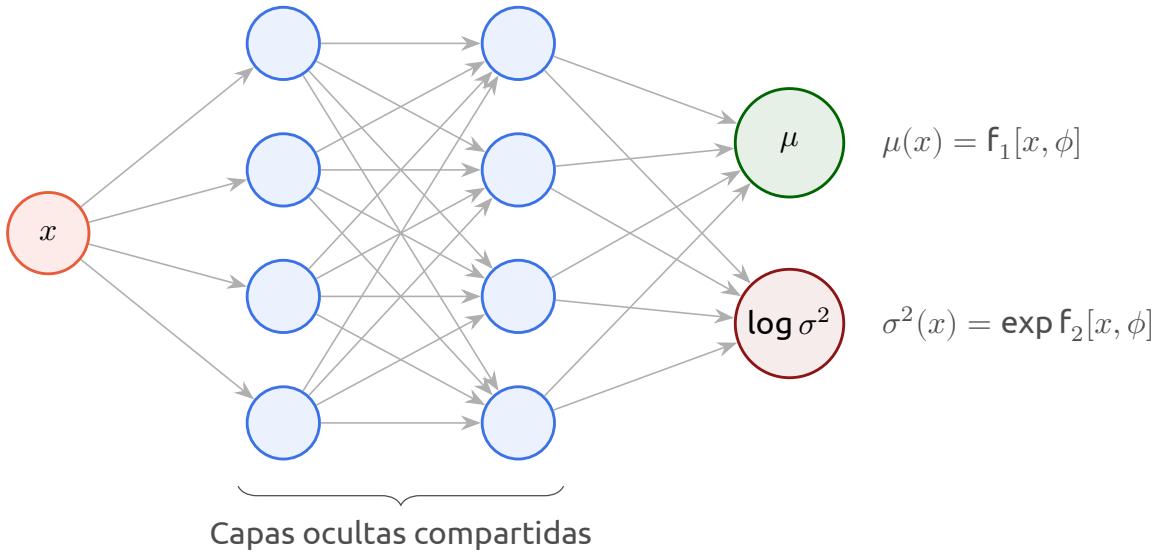
$$\mu(\mathrm{x}) = \mathsf{f}_1[\mathrm{x}, \phi], \qquad \sigma^2(\mathrm{x}) = \exp\left(\mathsf{f}_2[\mathrm{x}, \phi]\right),$$

y la pérdida es

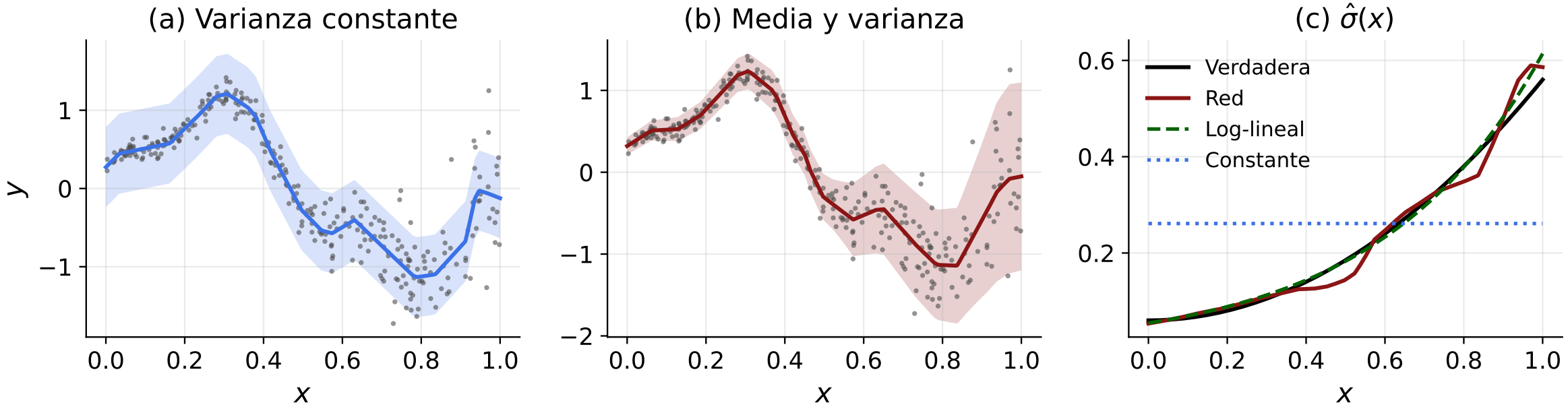
$$L[\phi] = \frac{1}{2}\sum_{i=1}^{n}\left[\mathsf{f}_{2,i} + \frac{(y_i - \mathsf{f}_{1,i})^2}{e^{\mathsf{f}_{2,i}}}\right] + \text{cte},$$

con $\mathsf{f}_{1,i} = \mathsf{f}_1[\mathrm{x}_i, \phi]$ y $\mathsf{f}_{2,i} = \mathsf{f}_2[\mathrm{x}_i, \phi]$.

Es la pérdida de la regresión heterocedástica con bases polinomiales para la media y para el logaritmo de la varianza, con la red en lugar de las dos bases.



Regresión heterocedástica, resultado



Son 300 datos simulados con desviación estándar $0{,}06 + 0{,}5\,x^2$. Red con dos capas ocultas de 64 unidades, $1\,500$ pasos con la varianza fija y $4\,500$ con la log verosimilitud completa. Sobre $20\,000$ observaciones nuevas, la log verosimilitud negativa media baja de $0{,}14$ a $-0{,}26$, y la cobertura del intervalo al 95 % para $x > 0{,}8$ sube de $0{,}72$ a $0{,}95$. El modelo con media polinomial y logaritmo de la varianza lineal en x logra $-0{,}28$.

Clasificación binaria

Con $y \in \{0, 1\}$ se usa la distribución de Bernoulli,

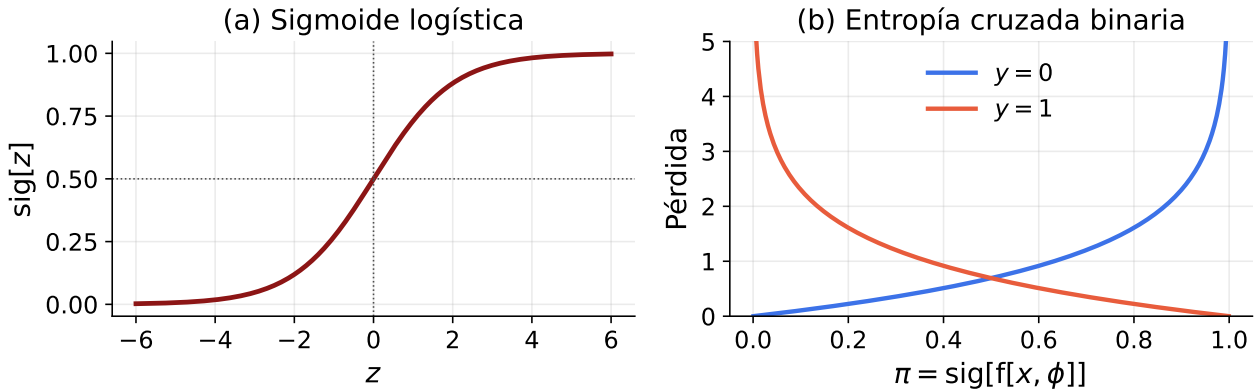
$$\Pr(y \mid \pi) = (1-\pi)^{1-y}\pi^{y}, \quad \pi \in [0, 1].$$

La red predice π a través de la sigmoide logística,

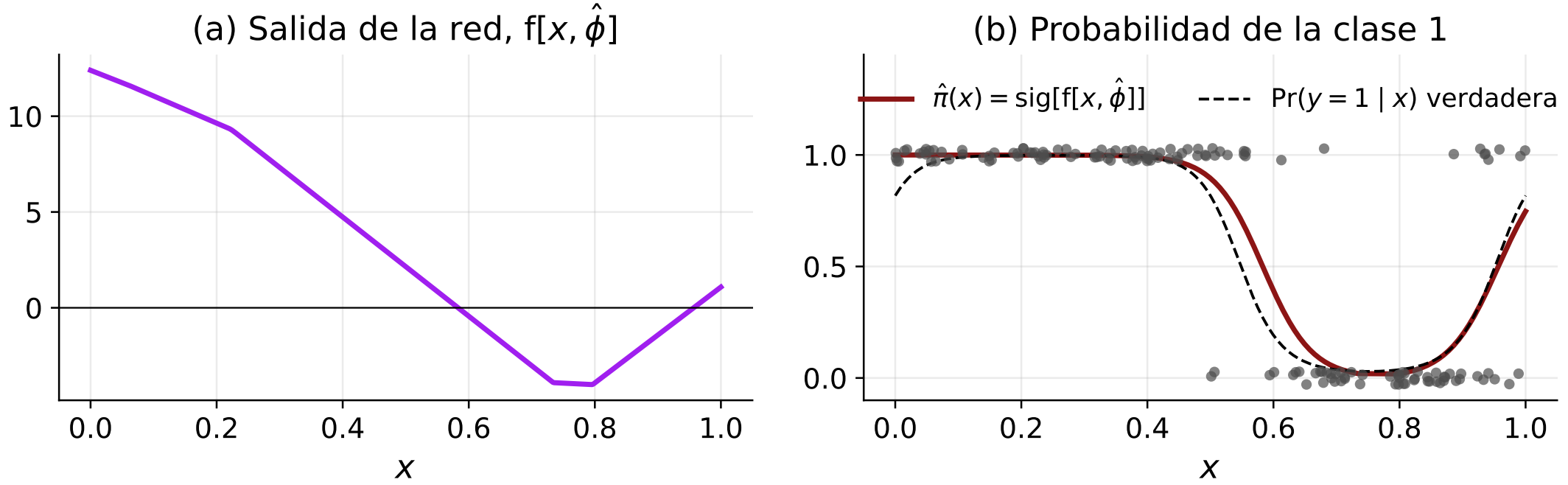
$$\pi = \text{sig}[\text{f}[\text{x}, \phi]], \quad \text{sig}[z] = \frac{1}{1+e^{-z}},$$

y la pérdida es la **entropía cruzada binaria**,

$$L[\phi] = -\sum_{i=1}^{n}\Big[(1-y_i)\log(1-\pi_i)+y_i \log \pi_i\Big].$$



Clasificación binaria, resultado



La red (dos capas de 32 unidades) produce un número real para cada x, y la sigmoide lo convierte en la probabilidad de la clase 1. Para clasificar se elige la clase más probable, $\hat{y} = 1\{\hat{\pi}(x) > 0{,}5\}$.

Clasificación en C clases

Con $y \in \{1, \ldots, C\}$ se usa la distribución categórica, $\Pr(y = c) = \pi_c$, con $\pi_c \geq 0$ y $\sum_c \pi_c = 1$.

La red tiene C salidas y la función **softmax** las convierte en probabilidades,

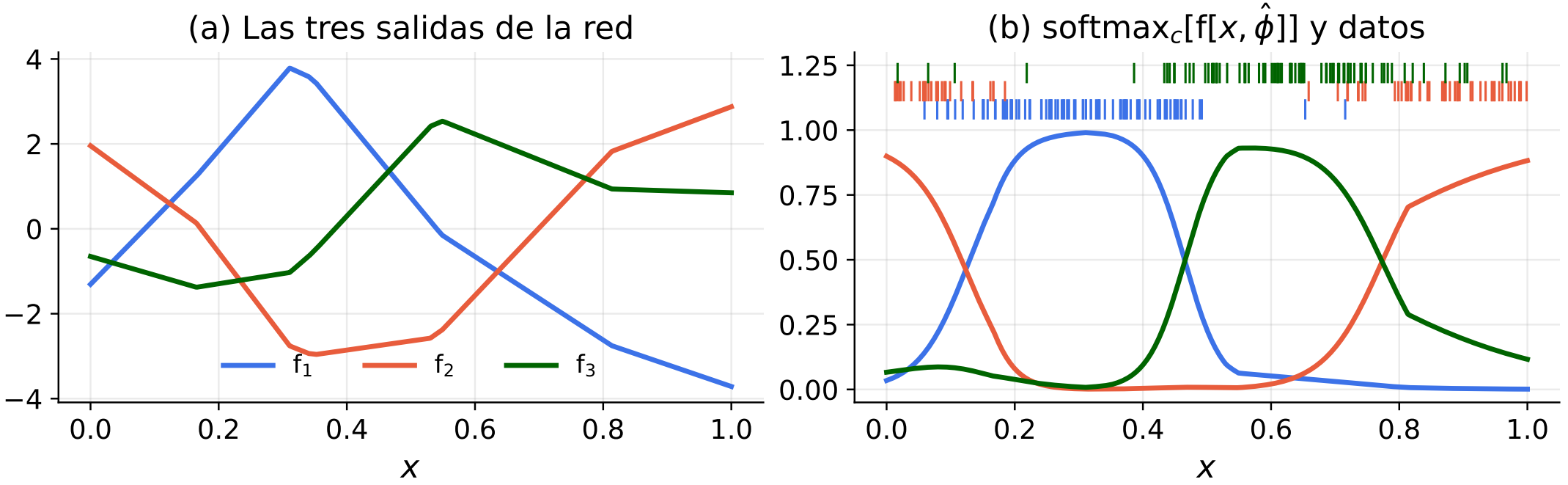
$$\pi_c = \text{softmax}_c[\mathrm{f}[\mathrm{x}, \phi]] = \frac{\exp(\mathrm{f}_c[\mathrm{x}, \phi])}{\sum_{c'=1}^{C} \exp(\mathrm{f}_{c'}[\mathrm{x}, \phi])}.$$

La pérdida es la **entropía cruzada multiclase**,

$$L[\phi] = -\sum_{i=1}^{n} \left[\mathrm{f}_{y_i}[\mathrm{x}_i, \phi] - \log \sum_{c'=1}^{C} \exp\left(\mathrm{f}_{c'}[\mathrm{x}_i, \phi]\right) \right].$$

El término log $\sum$ exp se calcula restando el máximo de las salidas antes de exponenciar, para evitar desbordes.

Clasificación en tres clases, resultado



Las salidas de la red (a) son números reales sin restricción. Después de la softmax (b) son probabilidades que suman uno en cada x. Las marcas en la parte superior son los 240 datos de entrenamiento, uno por fila de cada clase. La clase predicha es la de mayor probabilidad, $\hat{y} = \text{argmax}_c\, \pi_c(x)$.

Varias salidas y tipos de respuesta

Si la respuesta es un vector $\mathrm{y} = (y_1, \ldots, y_D)$ y se supone que sus componentes son independientes dado x,

$$\Pr(\mathrm{y} \mid \mathrm{f}[\mathrm{x}, \phi]) = \prod_{d=1}^{D} \Pr\left(y_d \mid \mathrm{f}_d[\mathrm{x}, \phi]\right),$$

$$L[\phi] = -\sum_{i=1}^{n}\sum_{d=1}^{D} \log \Pr\left(y_{id} \mid \mathrm{f}_d[\mathrm{x}_i, \phi]\right).$$

Las pérdidas de las distintas salidas se suman, aunque sean de tipos distintos. Para predecir la dirección y la intensidad del viento, por ejemplo, se suma la pérdida de una distribución de von Mises para la dirección y la de una exponencial para la intensidad.

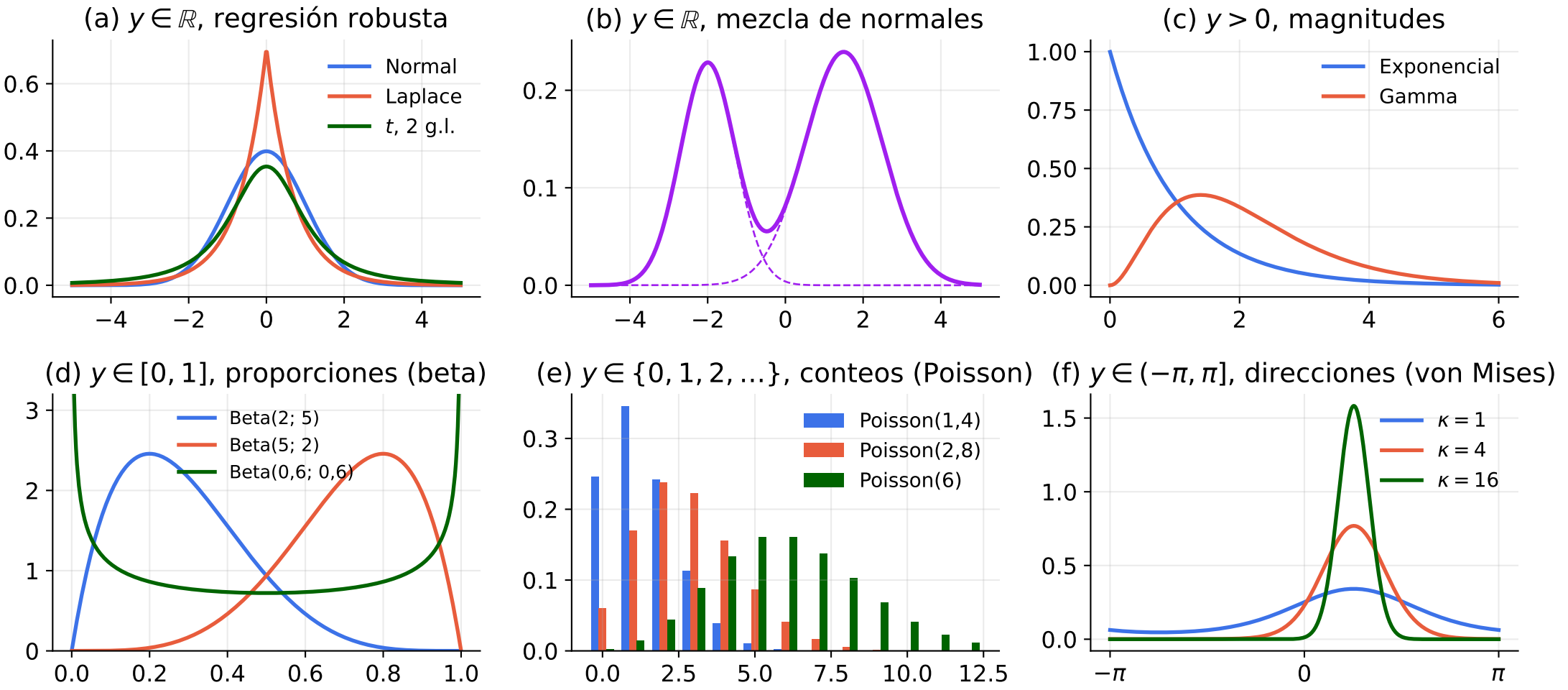
Si las salidas tienen escalas muy distintas, como la estatura en metros y el peso en kilos con varianza fija, una domina la suma. Estimar una varianza por salida, o estandarizar las respuestas, corrige el problema.

Distribuciones según el dominio de la respuesta

Tipo de dato	Dominio	Distribución	Uso
Continuo, no acotado	$y \in \mathbb{R}$	Normal	Regresión
Continuo, no acotado	$y \in \mathbb{R}$	Laplace o t	Regresión robusta
Continuo, no acotado	$y \in \mathbb{R}$	Mezcla de normales	Regresión multimodal
Continuo, positivo	$y \in \mathbb{R}^+$	Exponencial o gamma	Magnitudes, duraciones
Continuo, acotado	$y \in [0, 1]$	Beta	Proporciones
Continuo, multivariado	$\mathbf{y} \in \mathbb{R}^{D_o}$	Normal multivariada	Regresión multivariada
Continuo, circular	$y \in (-\pi, \pi]$	Von Mises	Direcciones
Discreto, binario	$y \in \{0, 1\}$	Bernoulli	Clasificación binaria
Discreto, acotado	$y \in \{1, \dots, C\}$	Categórica	Clasificación multiclase
Discreto, conteos	$y \in \{0, 1, 2, \dots\}$	Poisson	Conteos de eventos
Permutación	$\mathbf{y} \in \text{Perm}[1, \dots, C]$	Plackett–Luce	Ordenamientos

La regresión con la distribución de Laplace minimiza el error absoluto y estima la mediana condicional en lugar de la media.

Las distribuciones de la tabla



Entropía cruzada y verosimilitud

La divergencia de Kullback–Leibler entre la distribución empírica de los datos, $q(y) = \frac{1}{n}\sum_i \delta[y - y_i]$, y el modelo $\Pr(y \mid \boldsymbol{\eta})$ es

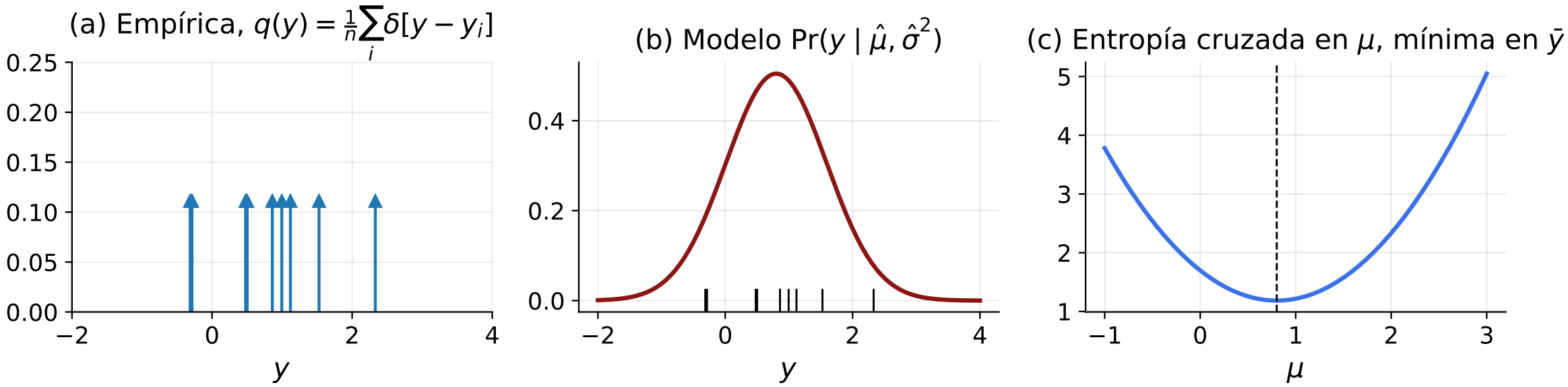
$$D_{\mathrm{KL}}[q \,\|\, \Pr] = \underbrace{\int q(y) \log q(y)\, dy}_{\text{no depende de } \boldsymbol{\eta}} - \underbrace{\int q(y) \log \Pr(y \mid \boldsymbol{\eta})\, dy}_{\text{entropía cruzada}}.$$

Minimizarla en $\boldsymbol{\eta}$ equivale a minimizar la entropía cruzada, y con la q empírica,

$$-\int \frac{1}{n}\sum_{i=1}^{n} \delta[y - y_i] \log \Pr(y \mid \boldsymbol{\eta})\, dy = -\frac{1}{n}\sum_{i=1}^{n} \log \Pr(y_i \mid \boldsymbol{\eta}).$$

Minimizar la entropía cruzada es minimizar la log verosimilitud negativa. Los dos nombres designan la misma pérdida.

Entropía cruzada, ilustración



La distribución empírica de nueve observaciones (a) es una suma de masas puntuales. El panel (b) muestra el modelo normal con los parámetros de máxima verosimilitud. Como función de μ, con σ fijo (c), la entropía cruzada es una parábola con mínimo en la media muestral.

Las pérdidas en PyTorch

Distribución	Pérdida	Entrada esperada
Normal, varianza fija	`nn.MSELoss`	Media
Normal, varianza predicha	`nn.GaussianNLLLoss`	Media y varianza positiva
Laplace	`nn.L1Loss`	Mediana
Bernoulli	`nn.BCEWithLogitsLoss`	Salida antes de la sigmoide
Categórica	`nn.CrossEntropyLoss`	Salidas antes de la softmax
Poisson	`nn.PoissonNLLLoss`	Logaritmo de la tasa

```python
def nll_heterocedastica(salida, y):       # salida: (n, 2) con media y log varianza
    media, log_var = salida[:, 0], salida[:, 1]
    return 0.5*(log_var + (y - media)**2*torch.exp(-log_var)).mean()
```

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

Ajustar el modelo

Entrenar es resolver

$$\hat{\phi} = \underset{\phi}{\text{argmin}}\ L[\phi], \qquad L[\phi] = \sum_{i=1}^{n} \ell_i, \quad \ell_i = -\log \Pr\left(\mathrm{y}_i \mid \mathrm{f}[\mathrm{x}_i, \phi]\right).$$

Salvo en casos lineales no hay solución cerrada. Los algoritmos de optimización usados en redes son iterativos.

1. Inicializar los parámetros ϕ_0, en general al azar.
2. Calcular el gradiente $\partial L / \partial \phi$ en el punto actual, que indica la dirección de mayor aumento de la pérdida.
3. Mover los parámetros en la dirección contraria y repetir hasta que la pérdida deje de bajar.

Descenso de gradiente

Con $\phi = (\phi_0, \dots, \phi_N)^\top$, el gradiente es el vector de derivadas parciales,

$$\frac{\partial L}{\partial \phi} = \left(\frac{\partial L}{\partial \phi_0}, \cdots, \frac{\partial L}{\partial \phi_N} \right)^\top,$$

y la actualización es

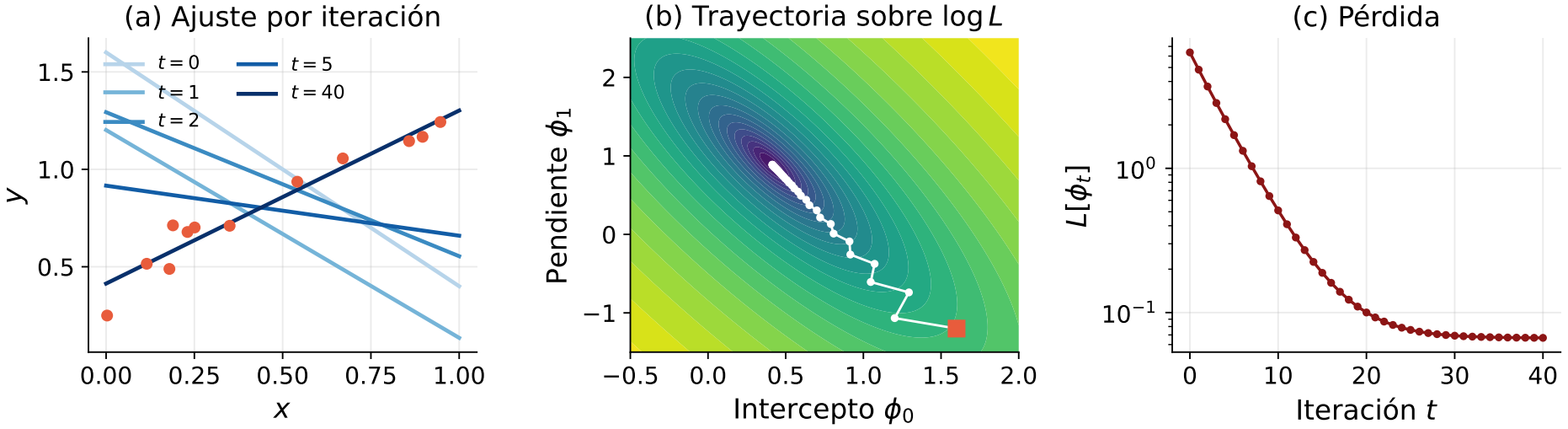
$$\phi_{t+1} \leftarrow \phi_t - \alpha \left. \frac{\partial L}{\partial \phi} \right|_{\phi_t}.$$

El escalar $\alpha > 0$ es la **tasa de aprendizaje**. Si es fija, es un hiperparámetro. Alternativamente, en cada paso se puede buscar el α que más reduce la pérdida en la dirección del gradiente, una búsqueda lineal, que en redes grandes resulta demasiado costosa.

Ejemplo, la regresión lineal

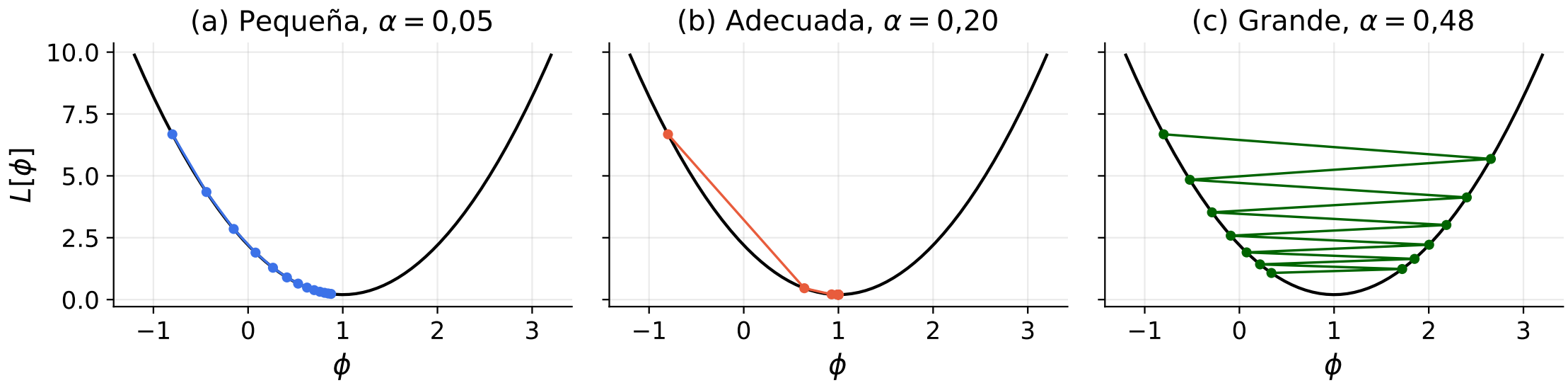
Con f$[x, \phi] = \phi_0 + \phi_1 x$ y pérdida cuadrática, $\ell_i = (\phi_0 + \phi_1 x_i - y_i)^2$, el gradiente es la suma de las contribuciones de cada observación,

$$\frac{\partial L}{\partial \phi} = \sum_{i=1}^{n} \frac{\partial \ell_i}{\partial \phi}, \qquad \frac{\partial \ell_i}{\partial \phi} = \begin{pmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{pmatrix}.$$



La pérdida es convexa, y el descenso de gradiente llega al mínimo desde cualquier punto de partida.

La tasa de aprendizaje



Sobre $L[\phi] = 2(\phi - 1)^2 + 0{,}2$, cuya segunda derivada vale 4. Con α pequeño la convergencia es lenta. Con α cercano a $1/4$ basta un paso. Con α entre $1/4$ y $2/4$ los pasos sobrepasan el mínimo y oscilan, y con $\alpha > 2/4$ divergen.

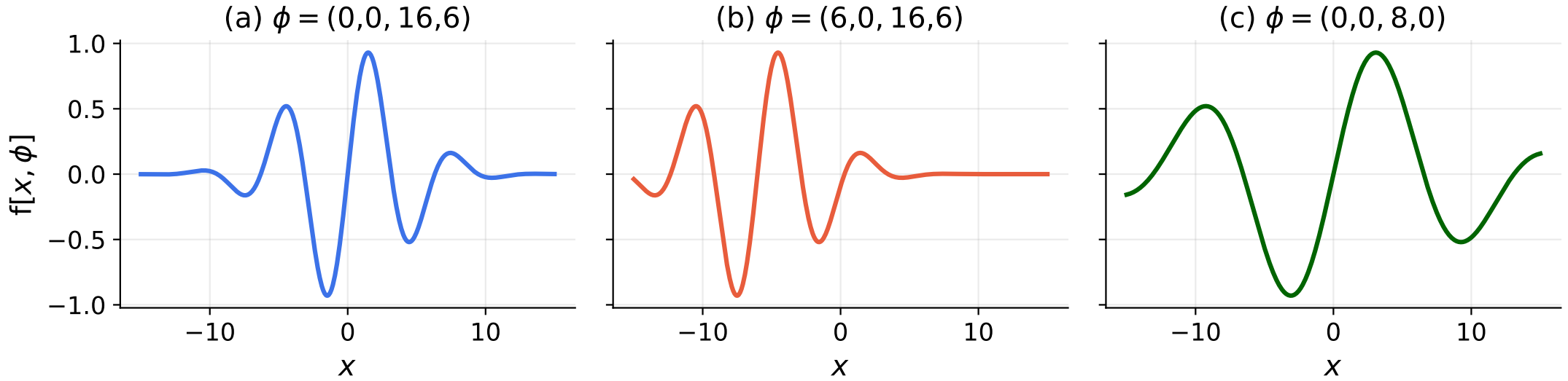
En general, sobre una pérdida cuadrática el descenso de gradiente converge si $\alpha < 2/\lambda_{\text{máx}}$, donde $\lambda_{\text{máx}}$ es el mayor valor propio del hessiano.

El modelo de Gabor

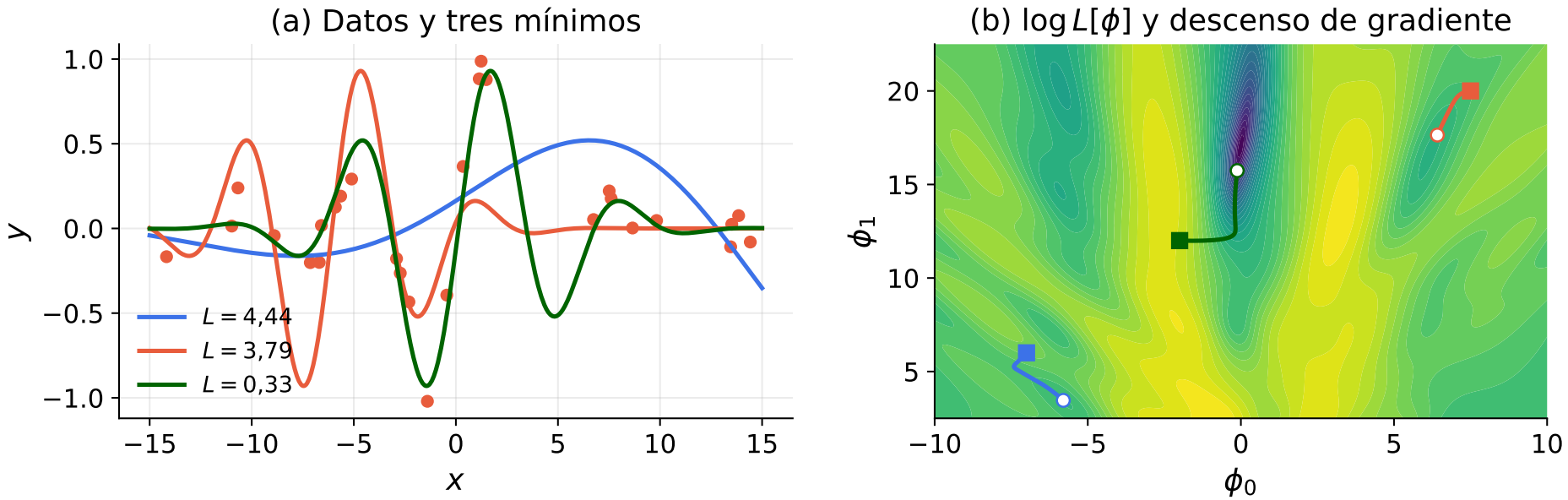
Las pérdidas de las redes no son convexas. Un modelo de dos parámetros permite verlo,

$$\mathrm{f}[x, \phi] = \sin(\phi_0 + 0{,}06\,\phi_1 x)\ \exp\left(-\frac{(\phi_0 + 0{,}06\,\phi_1 x)^2}{32}\right),$$

donde ϕ_0 desplaza la función y ϕ_1 la comprime o estira.



Mínimos locales y puntos silla



Con 28 datos generados con $\phi = (0; 16{,}6)$ y ruido normal, la pérdida tiene varios mínimos. El descenso de gradiente desde tres puntos de partida (cuadrados) llega a tres mínimos distintos (círculos blancos), y sólo uno es el global.

El punto final depende del inicio, y el algoritmo no tiene forma de saber si el mínimo alcanzado es global. Entre los mínimos hay **puntos silla**, donde el gradiente se anula sin que la pérdida sea mínima en todas las direcciones.

Descenso de gradiente estocástico

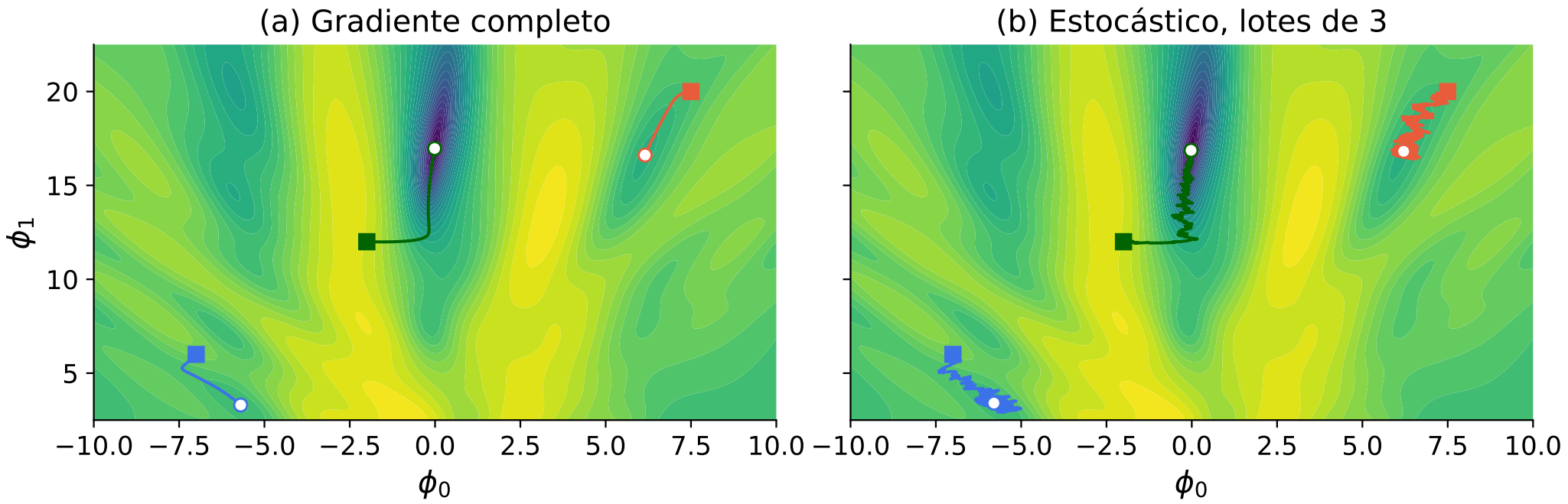
En cada iteración t el descenso de gradiente estocástico (SGD) elige al azar, sin reposición, un subconjunto $\mathcal{B}_t$ de las observaciones, el **lote**, y se usa sólo su gradiente,

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i}{\partial \phi}\bigg|_{\phi_t}.$$

Una pasada completa por los datos es una **época**. El lote puede tener desde una observación hasta todas, el caso del descenso de gradiente completo.

- El gradiente del lote es un estimador insesgado del gradiente completo, escalado por $|\mathcal{B}|/n$.
- Cada iteración cuesta $|\mathcal{B}|$ evaluaciones y no n, y se pueden dar muchos pasos por época.
- El ruido del estimador puede ayudar a salir de puntos silla, y su efecto sobre la generalización se discute en la parte 6.

Descenso estocástico sobre el modelo de Gabor



Con lotes de tres observaciones la trayectoria es ruidosa pero sigue en promedio la del gradiente completo. En este ejemplo el ruido no alcanza para cambiar de cuenca, y los tres puntos de partida terminan en los mismos mínimos. Escapar de un mínimo local es posible con SGD, pero no está garantizado.

El calendario de la tasa de aprendizaje

SGD con tasa fija no converge a un punto, porque el gradiente de cada lote no se anula en el mínimo. La tasa se reduce a lo largo del entrenamiento.

- **Por escalones.** Se multiplica por un factor, por ejemplo $0,5$, cada cierto número de épocas.
- **Coseno.** Decrece suavemente desde $\alpha_{\text{máx}}$ hasta casi cero siguiendo medio periodo de un coseno.
- **Calentamiento.** Crece linealmente durante las primeras iteraciones, lo que estabiliza el inicio cuando la pérdida es grande y la curvatura desconocida.

Al comienzo, una tasa alta permite recorrer el espacio de parámetros y encontrar una región razonable. Al final, una tasa baja permite ajustar los parámetros dentro de esa región.

Momentum

Se actualiza con un promedio móvil de los gradientes de los lotes recientes,

$$\mathrm{m}_{t+1} \leftarrow \rho\, \mathrm{m}_t + (1-\rho) \sum_{i \in \mathcal{B}_t} \left. \frac{\partial \ell_i}{\partial \phi} \right|_{\phi_t},$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha\, \mathrm{m}_{t+1},$$

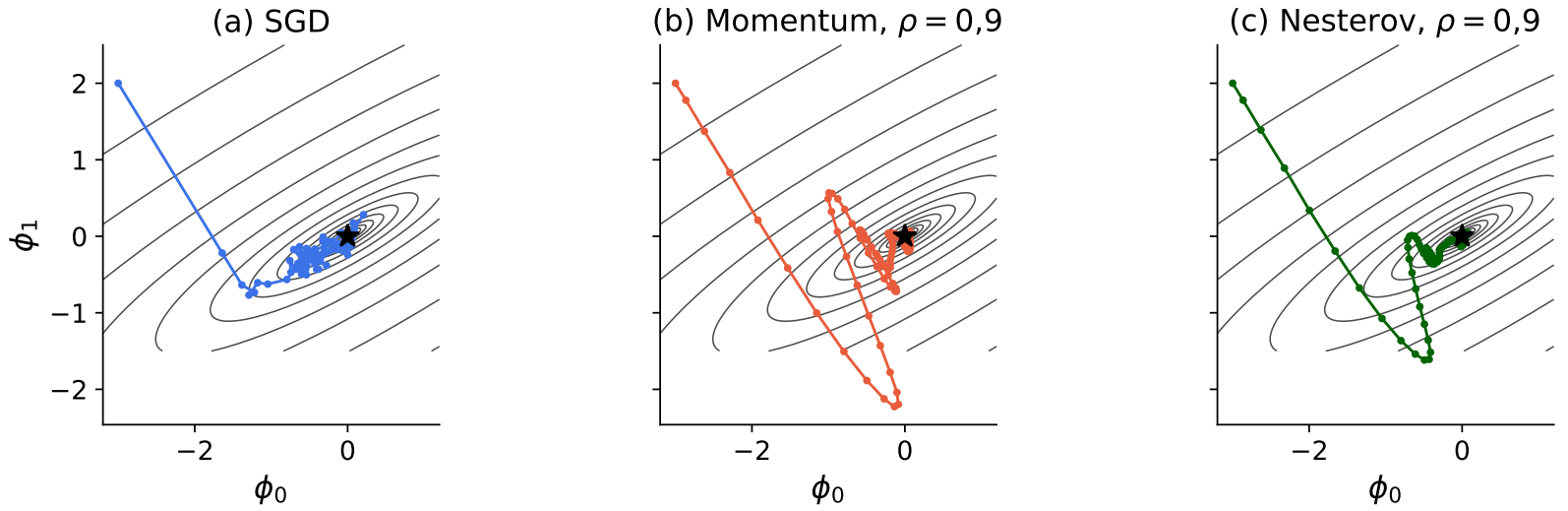
con $\rho \in [0, 1)$, típicamente $0{,}9$.

- En las direcciones donde los gradientes sucesivos coinciden, el promedio crece y el avance se acelera.
- En las direcciones donde alternan de signo, como al oscilar en un valle estrecho, se cancelan.
- El ruido de los lotes se promedia sobre unas $1/(1-\rho)$ iteraciones.

Momentum de Nesterov

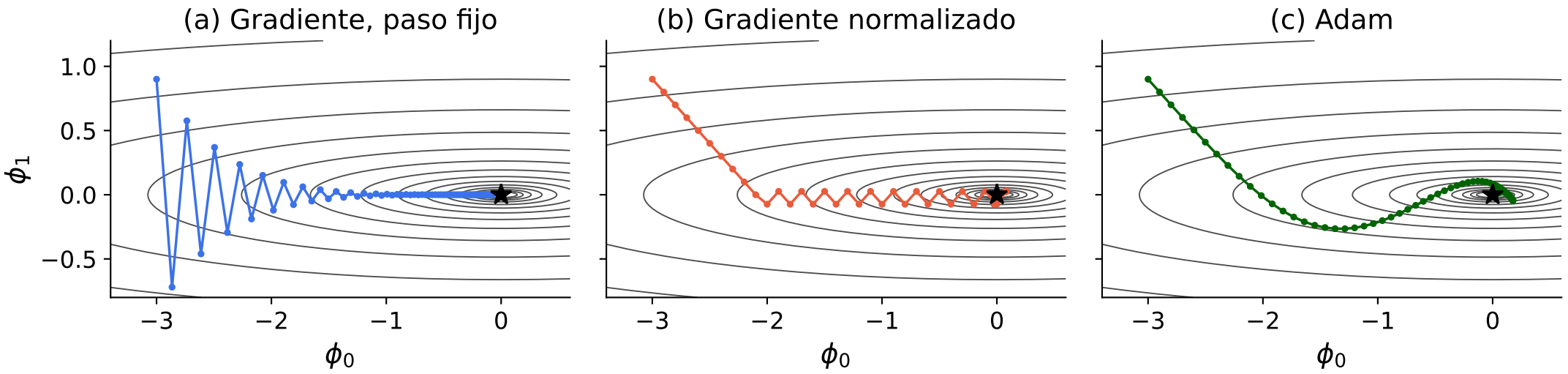
El momentum predice que el próximo paso irá aproximadamente a $\phi_t - \alpha\rho \mathrm{m}_t$, y el método de Nesterov evalúa allí el gradiente,

$$\mathrm{m}_{t+1} \leftarrow \rho\, \mathrm{m}_t + (1-\rho) \sum_{i \in \mathcal{B}_t} \left.\frac{\partial \ell_i}{\partial \phi}\right|_{\phi_t - \alpha\rho \mathrm{m}_t}, \qquad \phi_{t+1} \leftarrow \phi_t - \alpha\, \mathrm{m}_{t+1}.$$



Pérdida cuadrática con curvaturas 1 y 18 y gradiente con ruido. El momentum acelera pero sobrepasa el mínimo; Nesterov corrige el sobrepaso porque mira el gradiente adelante.

El problema de un paso común



Pérdida con curvaturas 1 y 40 en los dos ejes. (a) Con un paso común, el eje empinado oscila y el plano avanza lento, y no hay tasa que sirva para ambos. (b) Dividir cada componente del gradiente por su valor absoluto da pasos de igual tamaño en cada eje, pero el algoritmo no converge y termina oscilando alrededor del mínimo. (c) Adam normaliza con promedios móviles y converge.

Adam

Adam (estimación adaptativa de momentos; Kingma y Ba, 2015) mantiene promedios móviles del gradiente y de su cuadrado elemento a elemento,

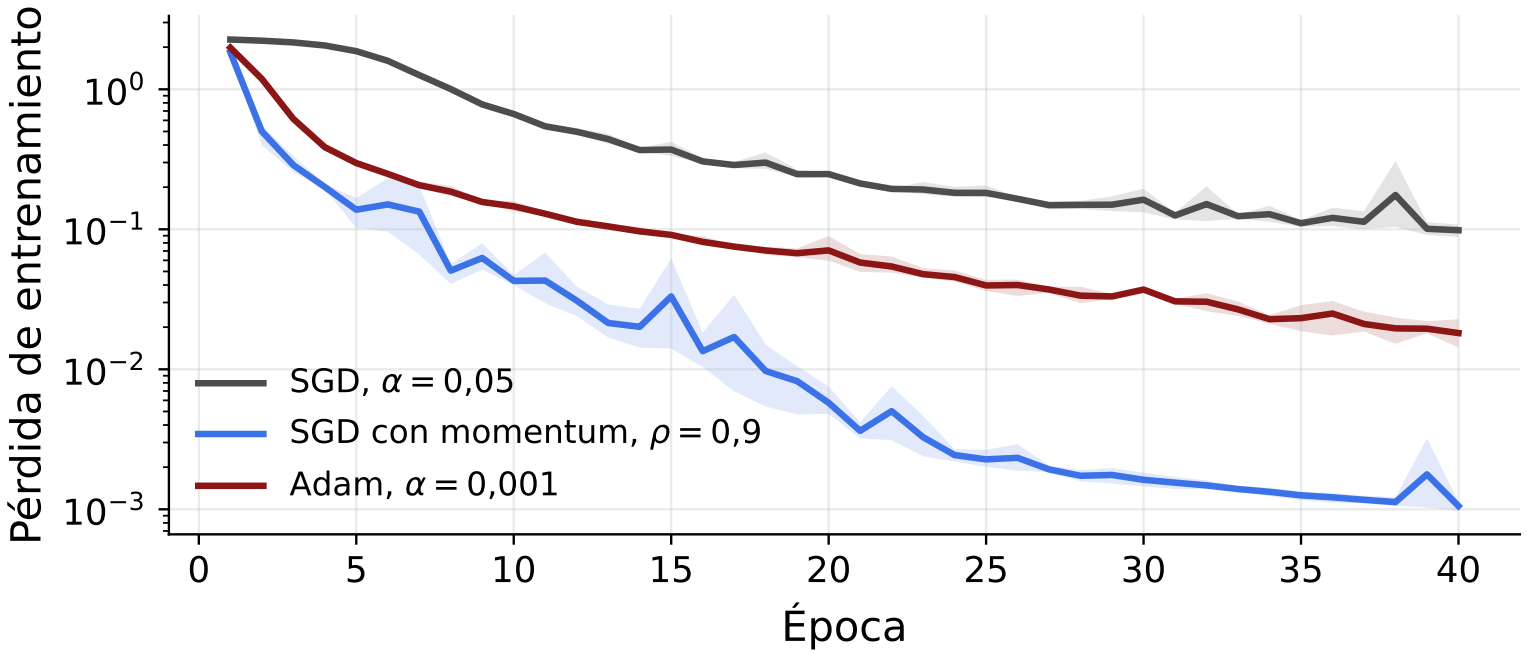
$$\mathrm{m}_{t+1} \leftarrow \rho_1\,\mathrm{m}_t + (1-\rho_1)\,\mathrm{g}_t, \quad \mathrm{v}_{t+1} \leftarrow \rho_2\,\mathrm{v}_t + (1-\rho_2)\,\mathrm{g}_t^2, \quad \mathrm{g}_t = \left.\frac{\partial L_{\mathcal{B}_t}}{\partial \phi}\right|_{\phi_t},$$

los corrige por su inicio en cero y normaliza el paso,

$$\tilde{\mathrm{m}} = \frac{\mathrm{m}_{t+1}}{1-\rho_1^{t+1}}, \qquad \tilde{\mathrm{v}} = \frac{\mathrm{v}_{t+1}}{1-\rho_2^{t+1}}, \qquad \phi_{t+1} \leftarrow \phi_t - \alpha\,\frac{\tilde{\mathrm{m}}}{\sqrt{\tilde{\mathrm{v}}}+\epsilon}.$$

- Los valores habituales son $\alpha = 10^{-3}$, $\rho_1 = 0{,}9$, $\rho_2 = 0{,}999$ y $\epsilon = 10^{-8}$.
- Cada parámetro tiene su propia escala de paso, del orden de α, sin importar la magnitud de su gradiente.
- Sin la corrección, en las primeras iteraciones m y v estarían sesgados hacia cero.

Comparación sobre una red real



Red con dos capas ocultas de 100 unidades sobre $1\,000$ imágenes de dígitos manuscritos, lotes de 32, tres semillas. El momentum reduce la pérdida de entrenamiento casi dos órdenes de magnitud más que SGD sin momentum en 40 épocas. Adam con su tasa por defecto queda entre ambos.

Recomendaciones prácticas

- Comenzar con Adam y $\alpha = 10^{-3}$, o con SGD con momentum $0{,}9$ y una tasa ajustada.
- La tasa de aprendizaje es el hiperparámetro más importante. Se elige en escala logarítmica, por ejemplo entre 10^{-4} y 10^{-1}, mirando la pérdida de validación.
- Una pérdida que crece o produce valores no numéricos indica una tasa demasiado alta. Una pérdida que baja muy lentamente, una tasa demasiado baja.
- Lotes de 32 a 256 son habituales. Lotes más grandes permiten tasas más altas y aprovechan mejor la unidad de procesamiento gráfico (GPU), pero dan menos pasos por época.
- Usar un calendario de la tasa, o al menos reducirla cuando la pérdida de validación se estanca.
- Estandarizar las entradas para que la pérdida esté mejor condicionada.

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

3. Funciones de pérdida y máxima verosimilitud

4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

Dos problemas del entrenamiento

El descenso de gradiente estocástico necesita, en cada iteración y para cada observación del lote,

$$\frac{\partial \ell_i}{\partial \boldsymbol{\beta}_k} \quad \text{y} \quad \frac{\partial \ell_i}{\partial \boldsymbol{\Omega}_k}, \qquad k = 0, 1, \ldots, K.$$

- **Cálculo.** Las redes actuales tienen entre 10^4 y 10^{12} parámetros, y el gradiente se calcula millones de veces. Calcular cada derivada por separado es inviable.
- **Inicialización.** Con parámetros iniciales mal escalados, las activaciones y los gradientes crecen o se anulan de capa en capa, y el entrenamiento no avanza.

La retropropagación resuelve el primero y una elección cuidadosa de la varianza inicial de los pesos, el segundo (capítulo 7 de Prince).

Un ejemplo de juguete

Una red con una unidad por capa y activaciones sin, exp y cos,

$$\mathsf{f}[x, \phi] = \beta_3 + \omega_3 \cos\Big[\beta_2 + \omega_2 \exp\big[\beta_1 + \omega_1 \sin[\beta_0 + \omega_0 x]\big]\Big],$$

con pérdida cuadrática $\ell_i = (\mathsf{f}[x_i, \phi] - y_i)^2$ y ocho parámetros.

Derivando directamente, por ejemplo,

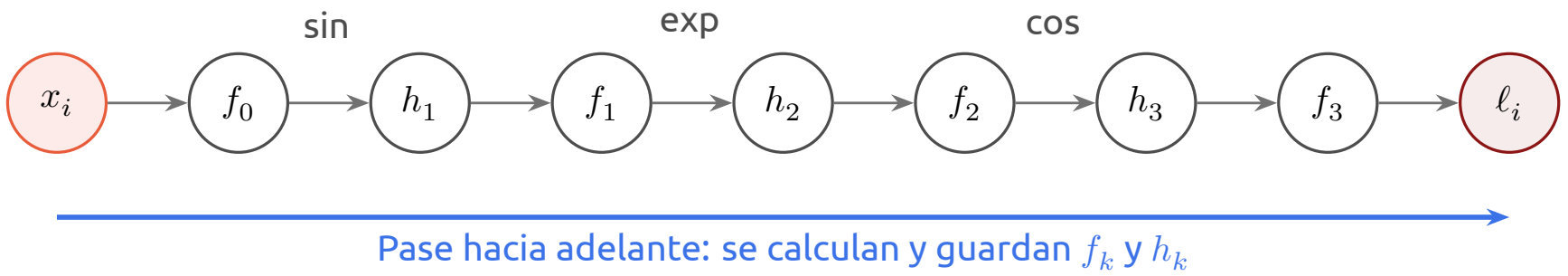
$$\begin{aligned}\frac{\partial \ell_i}{\partial \omega_0} = -2\Big(\beta_3 + \omega_3 \cos\big[\beta_2 + \omega_2 \exp[\beta_1 + \omega_1 \sin[\beta_0 + \omega_0 x_i]]\big] - y_i\Big) \\ \cdot\ \omega_1\omega_2\omega_3\, x_i \cos[\beta_0 + \omega_0 x_i] \exp\big[\beta_1 + \omega_1 \sin[\beta_0 + \omega_0 x_i]\big] \\ \cdot\ \sin\big[\beta_2 + \omega_2 \exp[\beta_1 + \omega_1 \sin[\beta_0 + \omega_0 x_i]]\big].\end{aligned}$$

Las ocho derivadas repiten los mismos factores muchas veces. La retropropagación los calcula una sola vez.

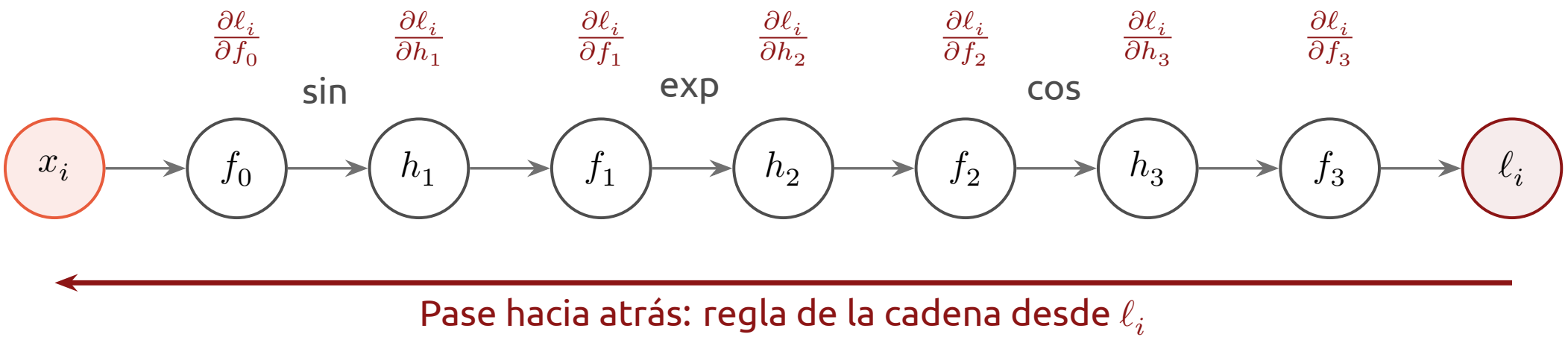
El pase hacia adelante

$$
\begin{aligned}
f_0 &= \beta_0 + \omega_0 x_i \\
h_1 &= \sin[f_0] \\
f_1 &= \beta_1 + \omega_1 h_1 \\
h_2 &= \exp[f_1] \\
f_2 &= \beta_2 + \omega_2 h_2 \\
h_3 &= \cos[f_2] \\
f_3 &= \beta_3 + \omega_3 h_3 \\
\ell_i &= (f_3 - y_i)^2
\end{aligned}
$$

La pérdida se escribe como una cadena de operaciones elementales. Se calculan en orden y se **guardan** todos los valores intermedios f_k y h_k.



El pase hacia atrás, primera etapa

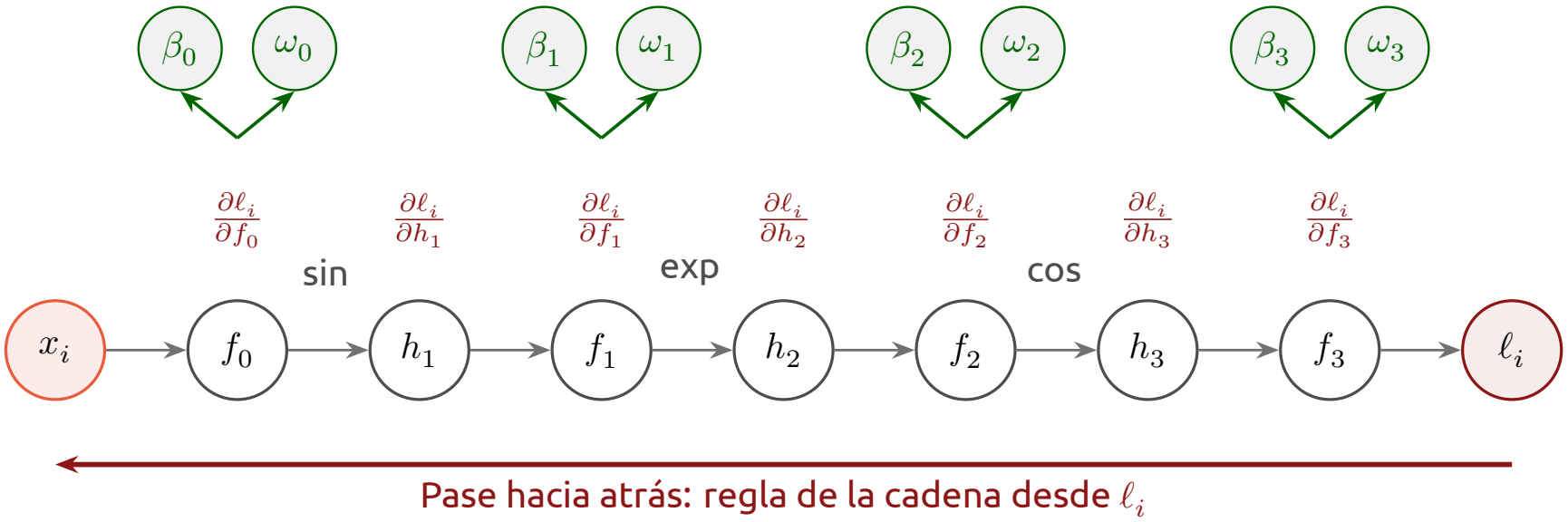


Se calculan las derivadas de ℓ_i respecto de los valores intermedios, en orden inverso, aplicando la regla de la cadena y reutilizando la derivada anterior,

$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i), \qquad \frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3} = \omega_3 \frac{\partial \ell_i}{\partial f_3}, \qquad \frac{\partial \ell_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2}\frac{\partial \ell_i}{\partial h_3} = -\sin[f_2]\frac{\partial \ell_i}{\partial h_3},$$

y así hasta $\partial \ell_i / \partial f_0$. Cada paso multiplica la derivada anterior por un factor local, que usa los valores guardados en el pase hacia adelante.

El pase hacia atrás, segunda etapa



Con $f_k = \beta_k + \omega_k h_k$ (y $h_0 = x_i$),

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial f_k}{\partial \beta_k}\frac{\partial \ell_i}{\partial f_k} = \frac{\partial \ell_i}{\partial f_k}, \qquad \frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial f_k}{\partial \omega_k}\frac{\partial \ell_i}{\partial f_k} = h_k \frac{\partial \ell_i}{\partial f_k}.$$

El efecto de cambiar un peso es proporcional al valor de la variable que ese peso multiplica.

Verificación numérica

Derivada	Retropropagación	Diferencias finitas	PyTorch
$\partial \ell / \partial \beta_0$	$2{,}280901$	$2{,}280901$	$2{,}280901$
$\partial \ell / \partial \omega_0$	$5{,}246071$	$5{,}246071$	$5{,}246071$
$\partial \ell / \partial \beta_1$	$-17{,}060467$	$-17{,}060467$	$-17{,}060467$
$\partial \ell / \partial \omega_1$	$-16{,}079299$	$-16{,}079299$	$-16{,}079299$
$\partial \ell / \partial \beta_2$	$-1{,}683058$	$-1{,}683058$	$-1{,}683058$
$\partial \ell / \partial \omega_2$	$-8{,}530233$	$-8{,}530233$	$-8{,}530233$
$\partial \ell / \partial \beta_3$	$0{,}744528$	$0{,}744528$	$0{,}744528$
$\partial \ell / \partial \omega_3$	$0{,}489469$	$0{,}489469$	$0{,}489469$

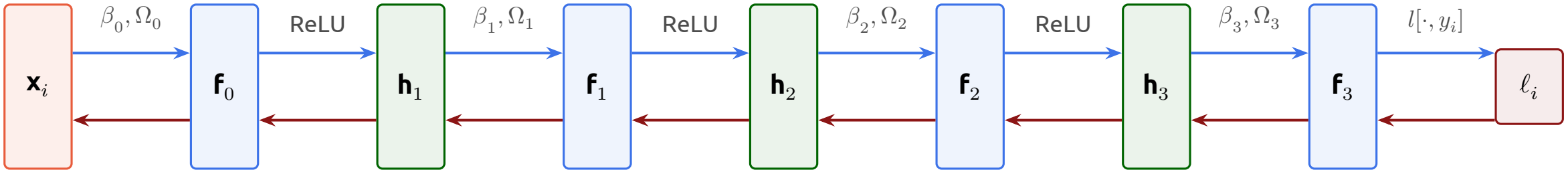
Con los valores del cuaderno 7.1 de Prince, $x_i = 2{,}3$ e $y_i = 2$. La retropropagación escrita a mano coincide con las diferencias finitas centradas, $(\ell(\phi + \Delta \mathrm{e}_j) - \ell(\phi - \Delta \mathrm{e}_j))/(2\Delta)$ con $\Delta = 10^{-6}$, y con la diferenciación automática de PyTorch.

La red con capas vectoriales

$$
\begin{aligned}
\mathbf{f}_0 &= \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i \\
\mathbf{h}_1 &= \mathbf{a}[\mathbf{f}_0] \\
\mathbf{f}_1 &= \boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1 \\
\mathbf{h}_2 &= \mathbf{a}[\mathbf{f}_1]
\end{aligned}
\qquad
\begin{aligned}
\mathbf{f}_2 &= \boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{h}_2 \\
\mathbf{h}_3 &= \mathbf{a}[\mathbf{f}_2] \\
\mathbf{f}_3 &= \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3 \\
\ell_i &= l[\mathbf{f}_3, y_i]
\end{aligned}
$$

La misma cadena, con vectores $\mathbf{f}_k$ (preactivaciones) y $\mathbf{h}_k$ (activaciones), sesgos $\boldsymbol{\beta}_k$ y matrices $\boldsymbol{\Omega}_k$. La activación es la ReLU y $l[\cdot, \cdot]$ es cualquiera de las pérdidas de la parte 3.

Adelante: se guardan $\mathbf{f}_k$ y $\mathbf{h}_k$



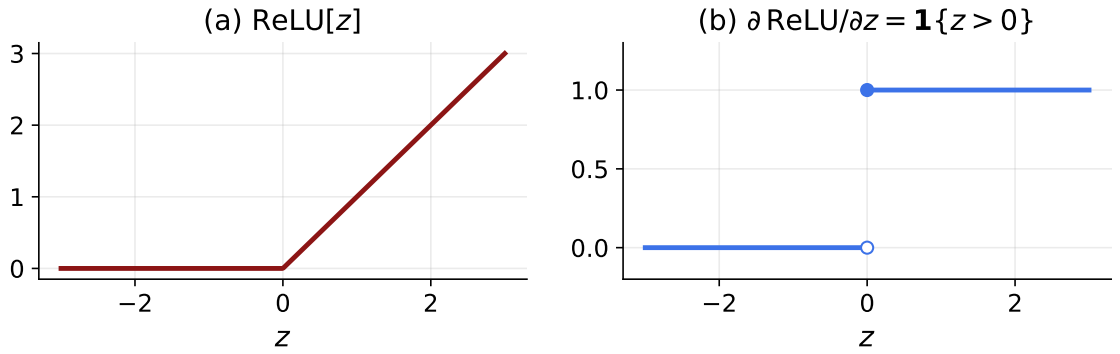
Atrás: se multiplica por $\Omega_k^\top$ y por $\mathbf{1}\{\mathbf{f}_{k-1} > 0\}$

La derivada de la ReLU

Cada activación depende sólo de su propia preactivación, de modo que $\partial \mathrm{h}_{k+1} / \partial \mathrm{f}_k$ es una matriz diagonal, con unos donde $\mathrm{f}_k > 0$ y ceros en el resto.

Multiplicar por esa matriz equivale a multiplicar elemento a elemento por el vector indicador $1\{\mathrm{f}_k > 0\}$, y así se implementa.

El gradiente sólo pasa por las unidades activas. Una unidad inactiva para todo el lote no recibe gradiente.



El pase hacia atrás en forma matricial

La regla de la cadena entre preactivaciones consecutivas da

$$\frac{\partial \ell_i}{\partial \mathrm{f}_{k-1}} = \underbrace{\frac{\partial \mathrm{h}_k}{\partial \mathrm{f}_{k-1}}}_{\text{activación}} \underbrace{\frac{\partial \mathrm{f}_k}{\partial \mathrm{h}_k}}_{\text{pesos}} \frac{\partial \ell_i}{\partial \mathrm{f}_k} = 1\{\mathrm{f}_{k-1} > 0\} \odot \left(\boldsymbol{\Omega}_k^\top \frac{\partial \ell_i}{\partial \mathrm{f}_k}\right).$$

y las derivadas respecto de los parámetros de cada capa son

$$\frac{\partial \ell_i}{\partial \boldsymbol{\beta}_k} = \frac{\partial \ell_i}{\partial \mathrm{f}_k}, \qquad \frac{\partial \ell_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial \ell_i}{\partial \mathrm{f}_k} \mathrm{h}_k^\top,$$

con $\mathrm{h}_0 = \mathrm{x}_i$. La segunda es un producto exterior, con las mismas dimensiones que $\boldsymbol{\Omega}_k$.

Hacia atrás se alternan dos operaciones, multiplicar por la traspuesta de la matriz de pesos y anular las componentes de las unidades inactivas, usando los f_{k-1} guardados.

El algoritmo

Pase hacia adelante. Calcular y guardar

$$\mathrm{f}_0 = \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathrm{x}_i, \qquad \mathrm{h}_k = \mathrm{a}[\mathrm{f}_{k-1}], \quad \mathrm{f}_k = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathrm{h}_k, \qquad k = 1, \ldots, K.$$

Pase hacia atrás. Partir de $\partial \ell_i / \partial \mathrm{f}_K$, que depende sólo de la pérdida, y para $k = K, K-1, \ldots, 1$ calcular

$$\frac{\partial \ell_i}{\partial \boldsymbol{\beta}_k} = \frac{\partial \ell_i}{\partial \mathrm{f}_k}, \qquad \frac{\partial \ell_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial \ell_i}{\partial \mathrm{f}_k} \mathrm{h}_k^\top, \qquad \frac{\partial \ell_i}{\partial \mathrm{f}_{k-1}} = 1\{\mathrm{f}_{k-1} > 0\} \odot \left(\boldsymbol{\Omega}_k^\top \frac{\partial \ell_i}{\partial \mathrm{f}_k} \right).$$

Terminar con $\partial \ell_i / \partial \boldsymbol{\beta}_0 = \partial \ell_i / \partial \mathrm{f}_0$ y $\partial \ell_i / \partial \boldsymbol{\Omega}_0 = (\partial \ell_i / \partial \mathrm{f}_0)\, \mathrm{x}_i^\top$.

Las derivadas de las observaciones del lote se suman para obtener el gradiente del paso de SGD. Con un lote, los vectores se convierten en matrices de $|\mathcal{B}|$ columnas y todo el cálculo es una sucesión de productos de matrices.

El costo de la retropropagación

La red de los dígitos, con dos capas ocultas de 100 unidades, tiene $17\,610$ parámetros. Sobre un lote de 64 imágenes, el pase hacia adelante y hacia atrás cuesta unas tres veces el pase hacia adelante sólo.

Las diferencias finitas necesitarían $2 \times 17\,610 = 35\,220$ pases hacia adelante por gradiente. La retropropagación calcula el gradiente completo con un costo que es un múltiplo pequeño y fijo del costo de evaluar la pérdida, cualquiera sea el número de parámetros.

El precio es la **memoria**. Todos los valores intermedios del pase hacia adelante se guardan hasta el pase hacia atrás, y en redes grandes esa memoria, más que el cálculo, limita el tamaño del lote y del modelo.

Diferenciación automática

En PyTorch o JAX no se escriben las derivadas. Cada operación elemental (producto de matrices, ReLU, suma, pérdida) conoce su derivada local, y la biblioteca registra durante el pase hacia adelante el **grafo de cálculo**, que luego recorre hacia atrás aplicando la regla de la cadena.

```
x = torch.tensor(2.3); y = 2.0
b0, w0, b1, w1, b2, w2, b3, w3 = [torch.tensor(v, requires_grad=True)
    for v in (1.0, 0.1, 2.0, -0.4, -3.0, 2.0, 0.4, 3.0)]
f = b3 + w3*torch.cos(b2 + w2*torch.exp(b1 + w1*torch.sin(b0 + w0*x)))
perdida = (f - y)**2
perdida.backward()          # pase hacia atrás
print(w0.grad)              # 5.2461, como en la tabla
```

El grafo puede tener ramificaciones y uniones. La única condición es que sea **acíclico**.

El problema de la inicialización

Con sesgos nulos y pesos $\Omega_{ij} \sim N(0, \sigma_\Omega^2)$, cada capa calcula $\mathrm{f}_k = \boldsymbol{\Omega}_k \, \mathrm{ReLU}[\mathrm{f}_{k-1}]$.

- Con σ_Ω^2 muy pequeño, cada preactivación es una suma de términos con pesos diminutos, y su magnitud decrece de capa en capa. Además, la ReLU anula la mitad de los valores.
- Con σ_Ω^2 muy grande, la magnitud crece de capa en capa.

En el pase hacia atrás se multiplica por $\boldsymbol{\Omega}_k^\top$ en cada capa, y el mismo argumento se aplica a los gradientes. Es el problema de los **gradientes que se desvanecen** o **explotan**, que impide entrenar redes profundas.

La varianza de las preactivaciones

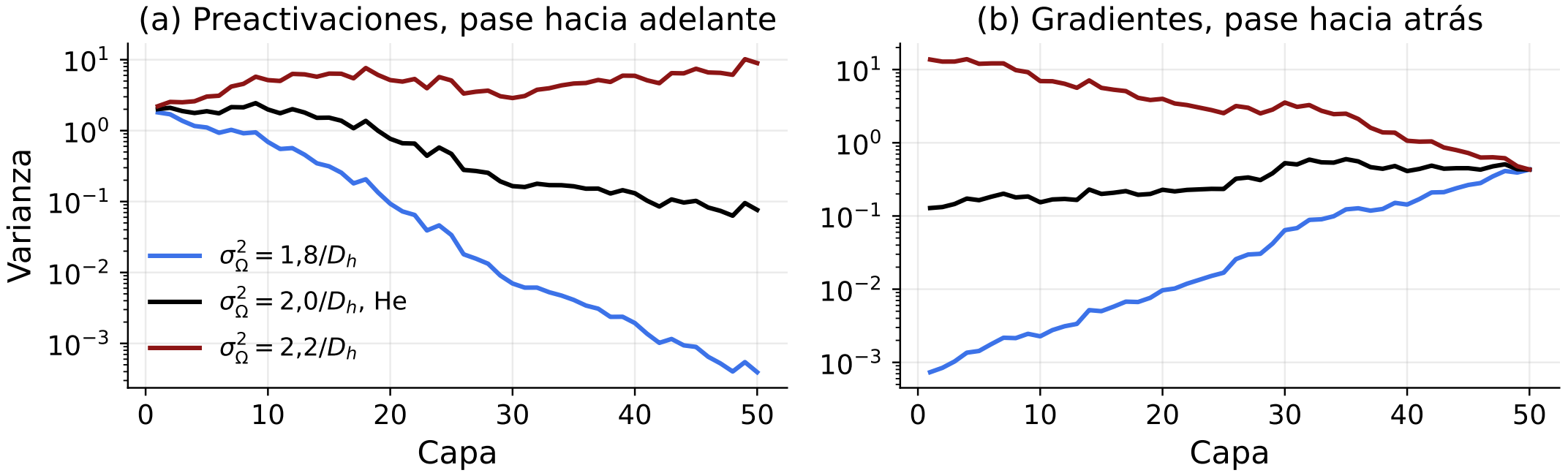
Sea $f'_i = \sum_{j=1}^{D_h} \Omega_{ij} h_j$ con $h_j = \text{ReLU}[f_j]$, los Ω_{ij} independientes de media cero y varianza σ_Ω^2, y los f_j simétricos con varianza σ_f^2. Entonces $\mathbb{E}[f'_i] = 0$ y

$$\sigma_{f'}^2 = \mathbb{E}\Big[\Big(\sum_j \Omega_{ij} h_j\Big)^2\Big] = \sum_{j=1}^{D_h} \mathbb{E}[\Omega_{ij}^2]\,\mathbb{E}[h_j^2] = D_h\,\sigma_\Omega^2\,\frac{\sigma_f^2}{2}.$$

El factor $1/2$ proviene de que la ReLU anula la mitad simétrica negativa, de modo que $\mathbb{E}[h_j^2] = \sigma_f^2/2$. Para que la varianza se conserve de capa en capa,

$$\sigma_\Omega^2 = \frac{2}{D_h} \qquad \text{(inicialización de He; He et al., 2015).}$$

La inicialización de He, verificada



Red de 50 capas de 100 unidades. Con $\sigma_\Omega^2 = 2/D_h$ las varianzas se mantienen en el mismo orden de magnitud a lo largo de la red. Un error de 10 % en la varianza se acumula como $0{,}9^{50} \approx 0{,}005$ o $1{,}1^{50} \approx 117$.

Entrenamiento en PyTorch, el modelo

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
D_i, D_k, D_o = 64, 100, 10                          # entrada, ocultas, salida
modelo = nn.Sequential(nn.Linear(D_i, D_k), nn.ReLU(),
                       nn.Linear(D_k, D_k), nn.ReLU(),
                       nn.Linear(D_k, D_o))
def inicializa(capa):                                # inicialización de He
    if isinstance(capa, nn.Linear):
        nn.init.kaiming_normal_(capa.weight)
        nn.init.zeros_(capa.bias)
modelo.apply(inicializa)
perdida = nn.CrossEntropyLoss()                      # parte 3
optimizador = torch.optim.SGD(modelo.parameters(), lr=0.1, momentum=0.9)   # parte 4
calendario = torch.optim.lr_scheduler.StepLR(optimizador, step_size=10, gamma=0.5)
datos = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)
```

Entrenamiento en PyTorch, el ciclo

```
for epoca in range(50):
    modelo.train()
    for x_lote, y_lote in datos:
        optimizador.zero_grad()                    # borrar gradientes anteriores
        l = perdida(modelo(x_lote), y_lote)        # pase hacia adelante
        l.backward()                               # retropropagación, parte 5
        optimizador.step()                         # paso de SGD
    calendario.step()                              # reducir la tasa
    modelo.eval()
    with torch.no_grad():                          # validación, parte 6
        error_val = (modelo(X_val).argmax(1) != y_val).float().mean()
    print(f"época {epoca:3d}, error de validación {error_val:.3f}")
```

Cada línea del ciclo corresponde a una parte de la clase. Olvidar `zero_grad` es un error frecuente, porque

PyTorch acumula los gradientes en lugar de reemplazarlos.

Contenido

1. De la regresión lineal a la red poco profunda

2. Redes profundas

3. Funciones de pérdida y máxima verosimilitud

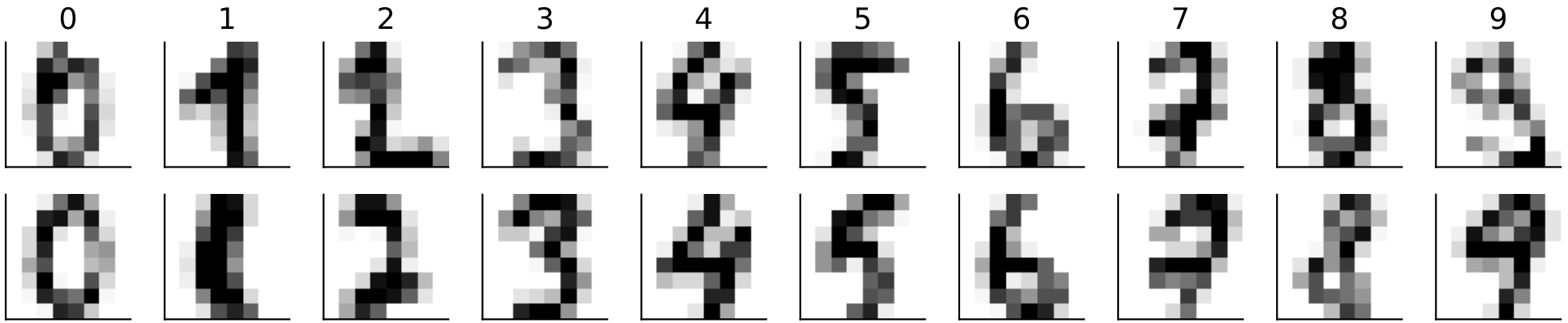
4. Entrenamiento: descenso de gradiente y sus variantes

5. Retropropagación e inicialización

6. Desempeño y regularización

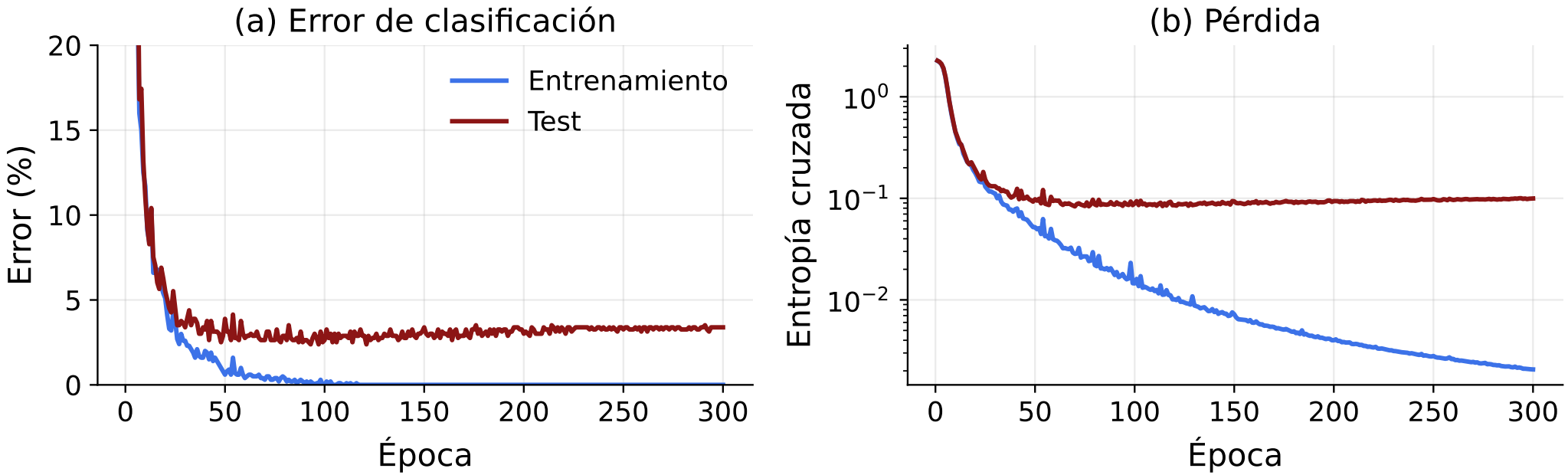
Medir el desempeño

Una red con suficiente capacidad suele ajustar perfectamente los datos de entrenamiento, lo que no implica que generalice. Las preguntas son cuánto se equivoca la red en datos nuevos y cómo reducir ese error (capítulos 8 y 9 de Prince).



Conjunto `digits` de scikit-learn: $1\,797$ imágenes de 8×8 píxeles de dígitos manuscritos, $D_i = 64$ y diez clases. Se usan $1\,000$ imágenes para entrenar y 797 para evaluar.

Entrenamiento y generalización



SGD con lotes de 50 y $\alpha = 0{,}1$ durante 300 épocas. El error de entrenamiento llega a cero hacia la época 100 y el de test se estabiliza en $3{,}4\,\%$. La pérdida de test alcanza su mínimo en la época 70 y después **sube**, aunque el error de clasificación no cambie.

Fuentes de error

El error esperado de test de la pérdida cuadrática en un punto x se descompone en tres términos,

$$\mathbb{E}_{\mathcal{D}}\Big[\mathbb{E}_y\big[L[x]\big]\Big] = \mathbb{E}_{\mathcal{D}}\Big[\big(\mathsf{f}[x,\phi[\mathcal{D}]] - \mathsf{f}_\mu[x]\big)^2\Big] + \big(\mathsf{f}_\mu[x] - \mu[x]\big)^2 + \sigma^2,$$

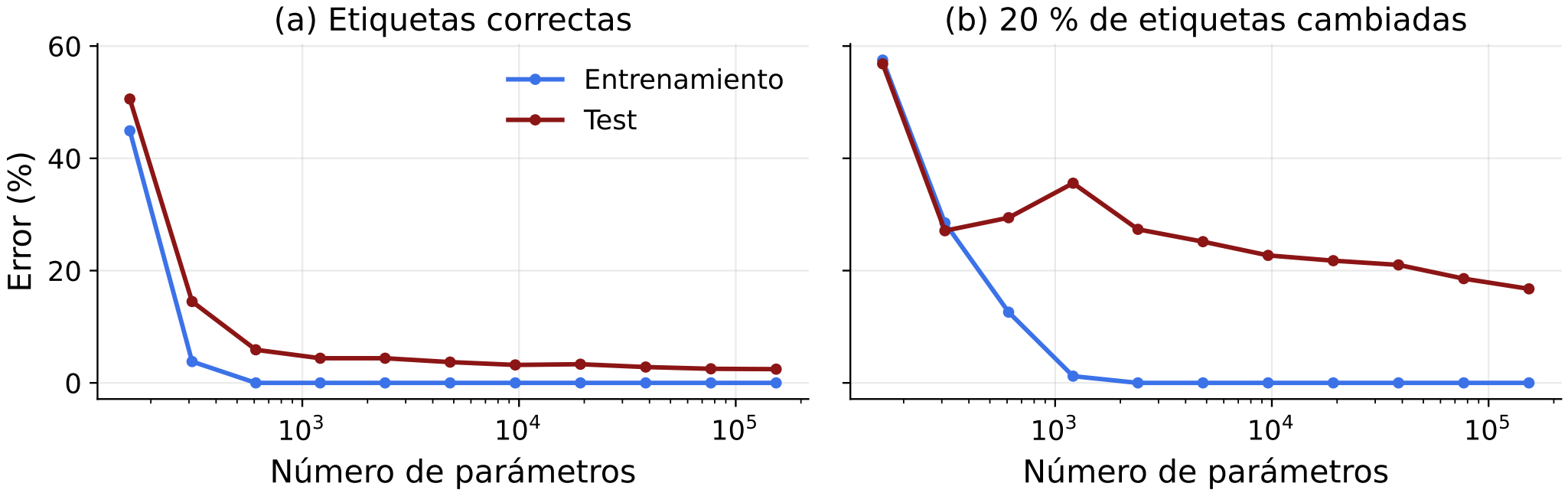
$$\text{varianza} \qquad\qquad \text{sesgo}^2 \qquad\qquad \text{ruido}$$

con $\mathsf{f}_\mu[x] = \mathbb{E}_{\mathcal{D}}[\mathsf{f}[x,\phi[\mathcal{D}]]]$.

- El ruido no se reduce con ningún modelo.
- El sesgo se reduce aumentando la capacidad, con más unidades o más capas.
- La varianza se reduce con más datos y con regularización.

En las redes aparece además el error de optimización, porque el entrenamiento no encuentra necesariamente el mínimo global.

Capacidad y doble descenso



Una capa oculta de ancho entre 2 y $2\,048, 500$ imágenes de entrenamiento, entrenada hasta memorizar. Con etiquetas correctas (a), el error de test decrece de forma monótona hasta $2,4$ %. Con 20 % de etiquetas cambiadas (b), el error de test sube de 27 % a 36 % cerca del ancho en que la red empieza a memorizar (16 unidades, unos $1\,200$ parámetros), y vuelve a bajar hasta 17 % con el ancho máximo.

Elegir la capacidad y los hiperparámetros

El número de capas, el ancho, la tasa de aprendizaje, el tamaño de lote y la intensidad de la regularización son hiperparámetros. Se eligen con el siguiente protocolo.

1. **Entrenar** varios modelos con distintos hiperparámetros sobre los datos de entrenamiento.
2. **Validar** cada uno sobre un conjunto de validación separado.
3. **Elegir** el de menor error de validación.
4. **Evaluar** sólo ese modelo, una vez, sobre el conjunto de test.

En redes, la validación cruzada de k pliegues suele ser demasiado costosa y se usa un único conjunto de validación. La búsqueda se hace al azar o con métodos que aprovechan los ensayos anteriores, como la optimización bayesiana, más eficientes que la rejilla cuando hay muchos hiperparámetros (Bergstra y Bengio, 2012).

Regularización explícita

La regularización agrega a la pérdida un término que prefiere ciertos parámetros,

$$\hat{\phi} = \underset{\phi}{\text{argmin}} \left[\sum_{i=1}^{n} \ell_i + \lambda \, \text{g}[\phi] \right].$$

La regularización tiene una lectura probabilística. Con una distribución a priori $\text{Pr}(\phi)$, el criterio de **máximo a posteriori**

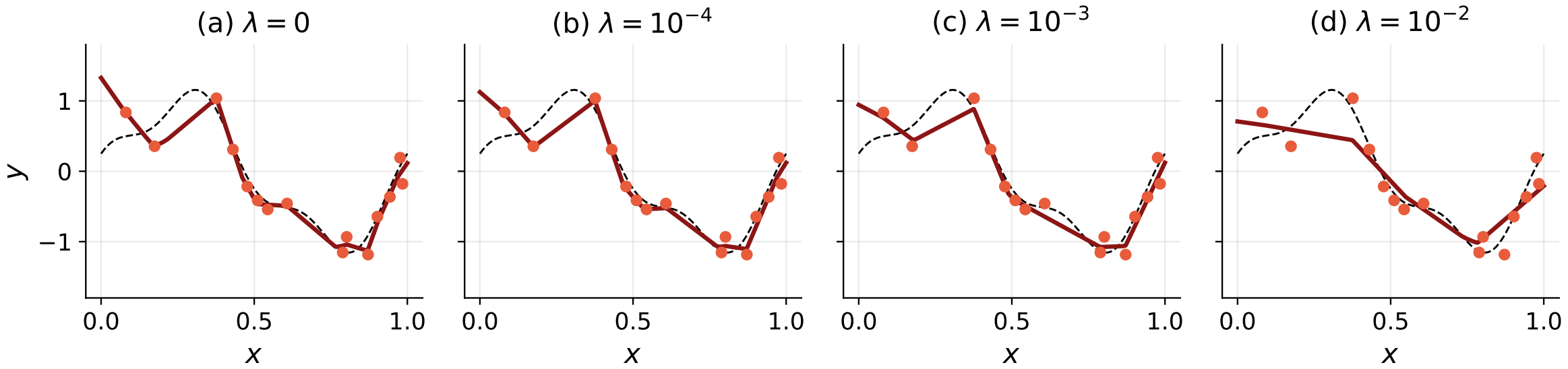
$$\hat{\phi} = \underset{\phi}{\text{argmax}} \left[\text{Pr}(\phi) \prod_{i=1}^{n} \text{Pr}\left(\text{y}_i \mid \text{f}[\text{x}_i, \phi]\right) \right]$$

equivale al anterior con $\lambda \, \text{g}[\phi] = -\log \text{Pr}(\phi)$.

En aprendizaje automático, regularización designa además cualquier estrategia que reduzca la brecha entre el desempeño de entrenamiento y el de test, aunque no agregue un término a la pérdida.

Regularización L2

La más usada es la penalización cuadrática de los pesos, $\lambda \sum_j \phi_j^2$, que corresponde a un a priori normal de media cero. En la regresión lineal es ridge, y en redes se llama **decaimiento de pesos**. Se aplica a los pesos y no a los sesgos.



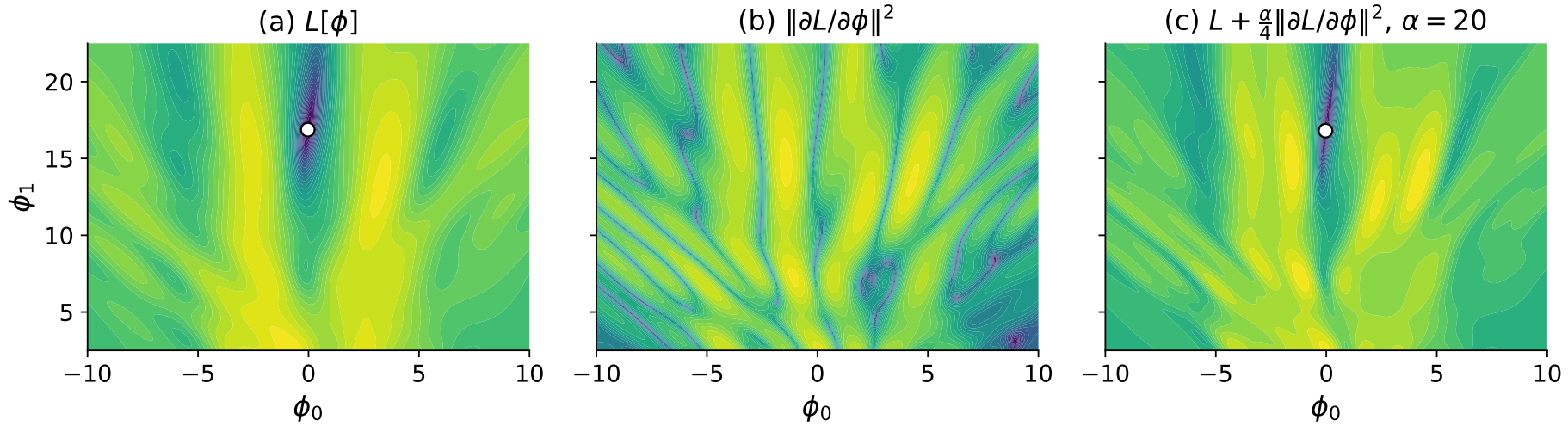
Quince observaciones, red con dos capas de 100 unidades. Al crecer λ la función se suaviza; con $\lambda = 10^{-2}$ se suaviza demasiado y pierde la estructura.

Regularización implícita del descenso de gradiente

El descenso de gradiente con paso finito α no sigue el flujo continuo $d\phi/dt = -\partial L/\partial \phi$.

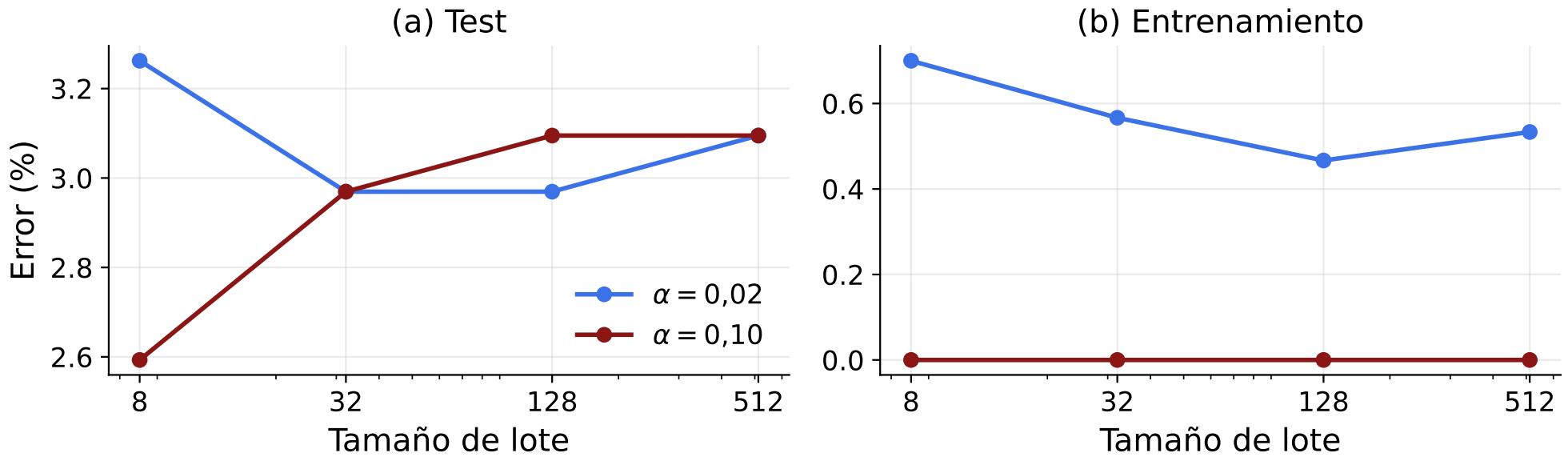
A primer orden en α, sigue el flujo de una pérdida modificada (Barrett y Dherin, 2021),

$$\tilde{L}_{\text{GD}}[\phi] = L[\phi] + \frac{\alpha}{4}\left\|\frac{\partial L}{\partial \phi}\right\|^2 .$$



El término adicional penaliza las regiones de gradiente grande y favorece los mínimos en zonas planas.

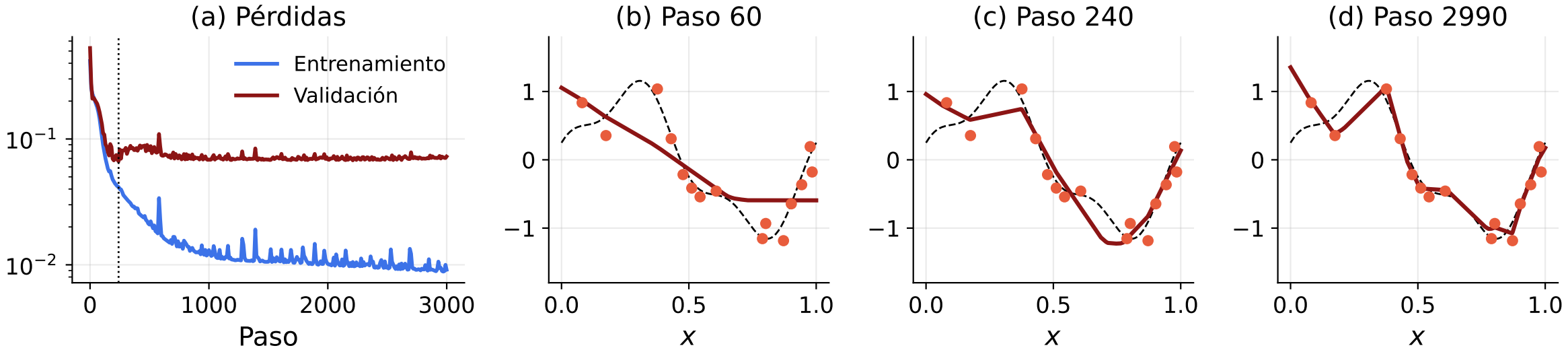
Tamaño de lote y tasa de aprendizaje



Dígitos, $6\,000$ pasos de SGD para todos los tamaños de lote, tres semillas. Con $\alpha = 0{,}1$ todas las redes memorizan el entrenamiento, y la de lotes de 8 tiene el menor error de test, $2{,}6$ % frente a $3{,}1$ % con lotes de 512. Con $\alpha = 0{,}02$ el error no cambia de forma monótona con el lote.

La tendencia general, lotes pequeños y tasas altas generalizan algo mejor, se observa aquí con diferencias de medio punto porcentual, del orden de la variación entre semillas.

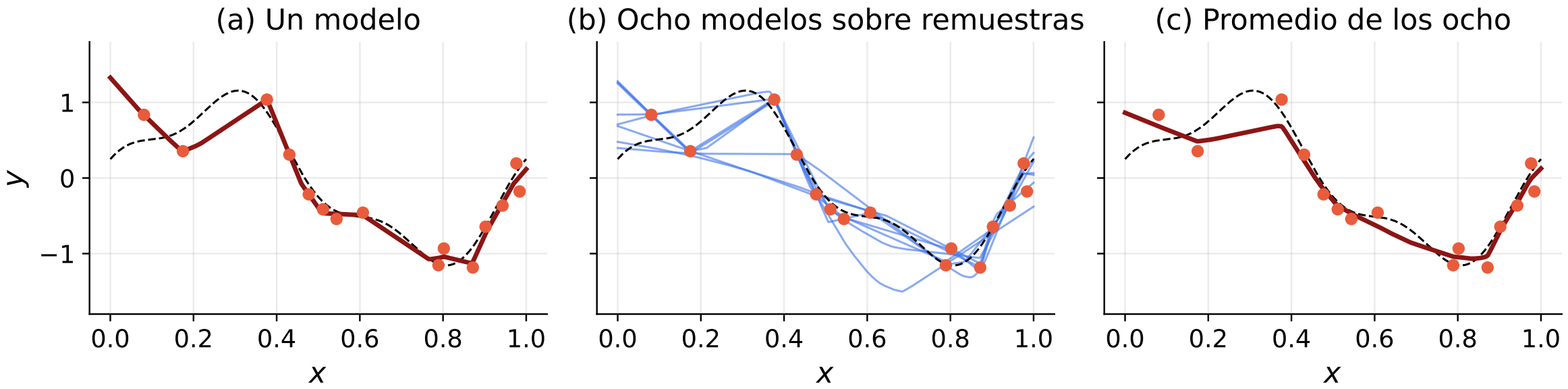
Parada temprana



Se detiene el entrenamiento antes de converger. Se guarda la red cada cierto número de pasos y se conserva la de menor pérdida de validación, aquí el paso 240. Con la red final la pérdida de validación es 9 % mayor.

En la regresión lineal, detener el descenso de gradiente en el paso t equivale aproximadamente a ridge con $\lambda \approx 1/(\alpha t)$.

Ensambles

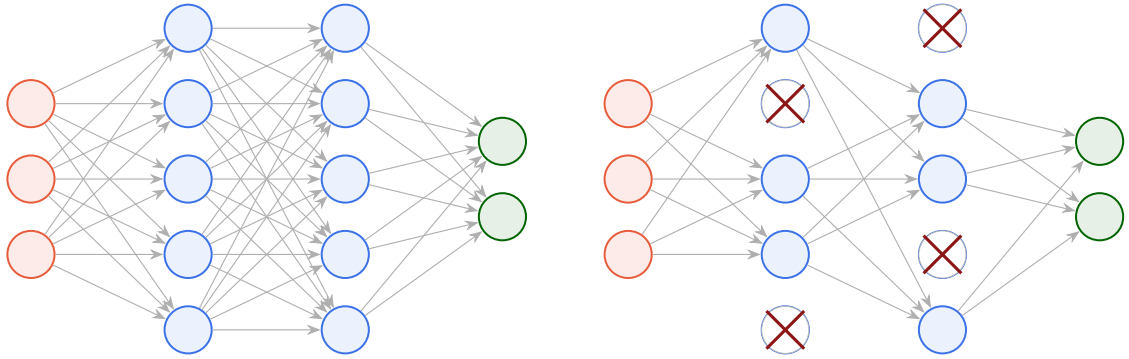


Se entrenan varias redes y se promedian sus predicciones, en regresión las salidas y en clasificación las probabilidades o las salidas antes de la softmax. La diversidad se obtiene con inicializaciones distintas, con hiperparámetros distintos o con remuestras bootstrap de los datos, el **bagging**. El promedio reduce la varianza y entrega una medida de incertidumbre, la dispersión entre modelos.

Dropout

En cada paso de SGD se anula cada unidad oculta con probabilidad p, con una máscara de Bernoulli nueva en cada pase hacia adelante y hacia atrás (Srivastava et al., 2014).

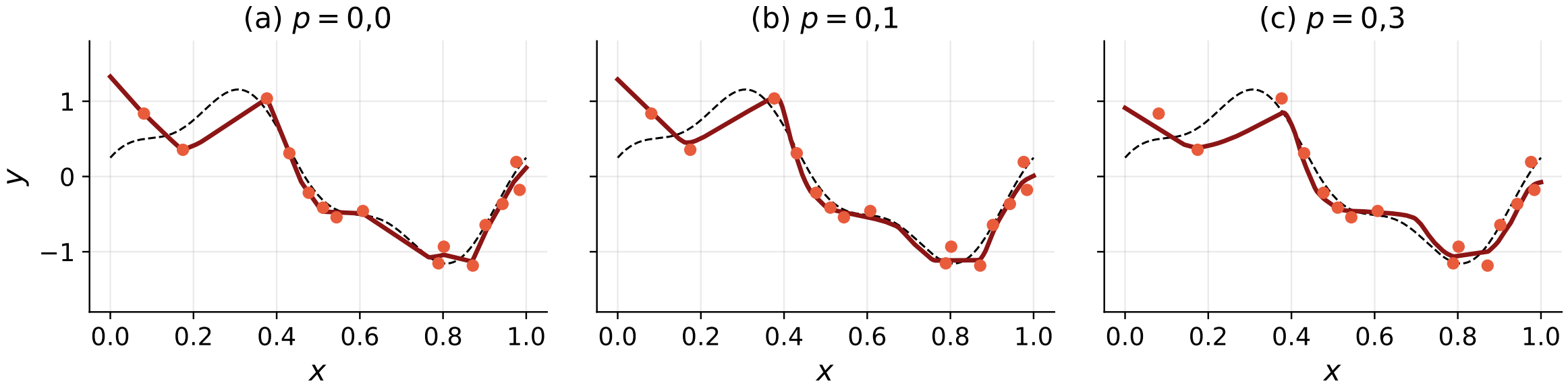
- La red no puede depender de unidades individuales y aprende representaciones redundantes.
- Equivale a entrenar un ensamble de 2^D subredes que comparten parámetros.
- En la evaluación se usan todas las unidades. Para que la escala coincida, durante el entrenamiento las activaciones que se conservan se multiplican por $1/(1-p)$.



(a) Red completa

(b) Una máscara de dropout

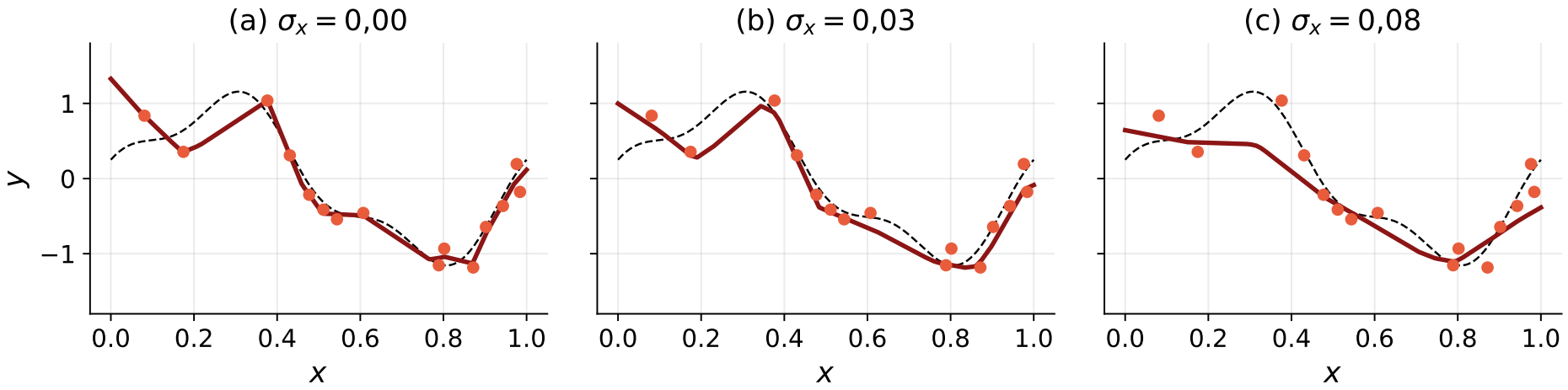
Dropout, efecto



Con $p = 0{,}1$ y $p = 0{,}3$ la función pierde algunos quiebres que sólo servían para pasar por puntos individuales.

Con pocos datos, un p alto suaviza en exceso.

Ruido en las entradas, en los pesos y en las etiquetas



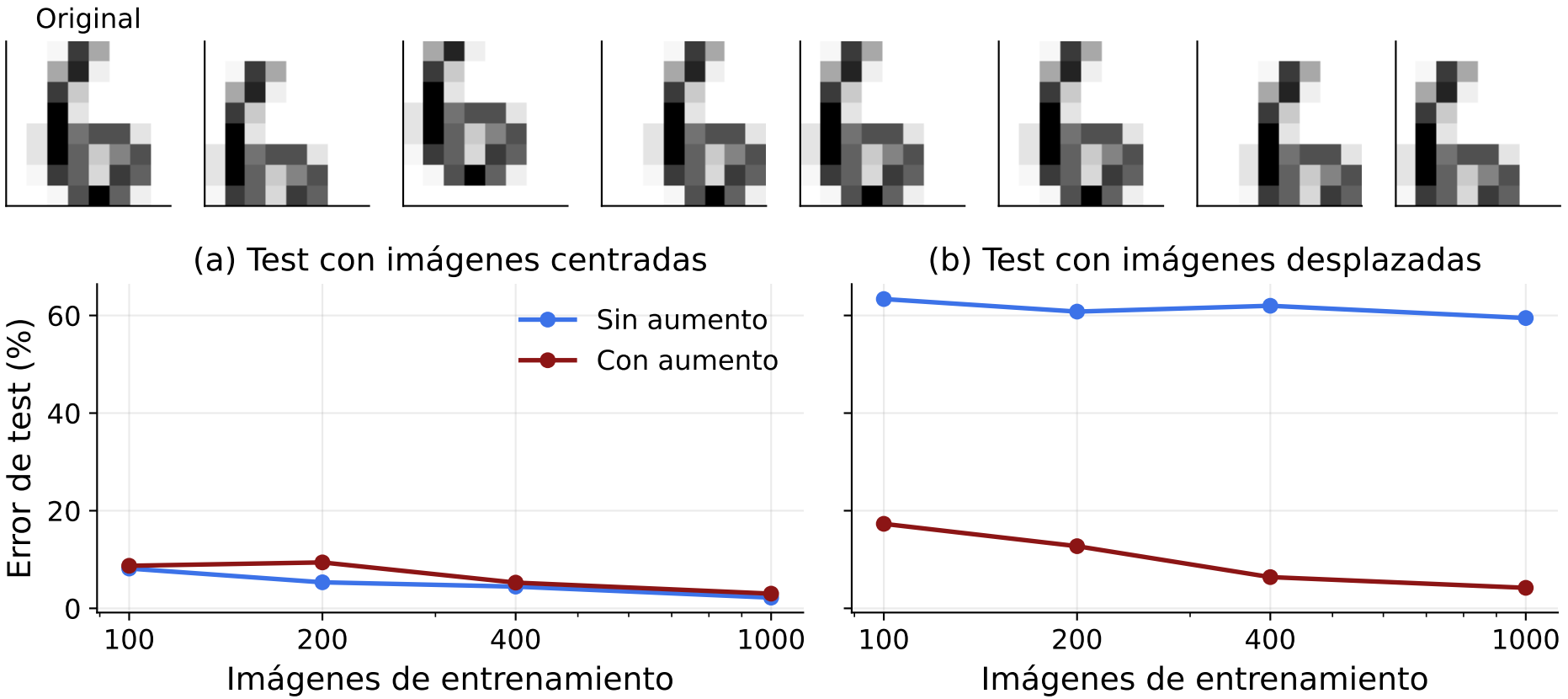
- **Entradas.** Sumar ruido de varianza σ_x^2 a cada lote suaviza la función. En regresión equivale a penalizar la derivada de la salida respecto de la entrada (Bishop, 1995). En la regresión lineal equivale a ridge.
- **Pesos.** Favorece mínimos anchos, donde perturbar los pesos cambia poco la pérdida.
- **Etiquetas.** El suavizado de etiquetas reemplaza la clase observada por una distribución que le asigna $1 - \gamma$ y reparte γ entre las demás, y evita que la red lleve sus salidas a valores extremos.

Comparación sobre diez conjuntos de datos

Método	ECM medio	Mejor que sin regularizar
Sin regularizar	$0{,}054$	–
L2, $\lambda = 10^{-3}$	$0{,}047$	7 de 10
Parada temprana, paso 240	$0{,}048$	8 de 10
Dropout, $p = 0{,}1$	$0{,}054$	4 de 10
Ruido en la entrada, $\sigma_x = 0{,}03$	$0{,}052$	5 de 10
Bagging, 8 modelos	$0{,}057$	2 de 10

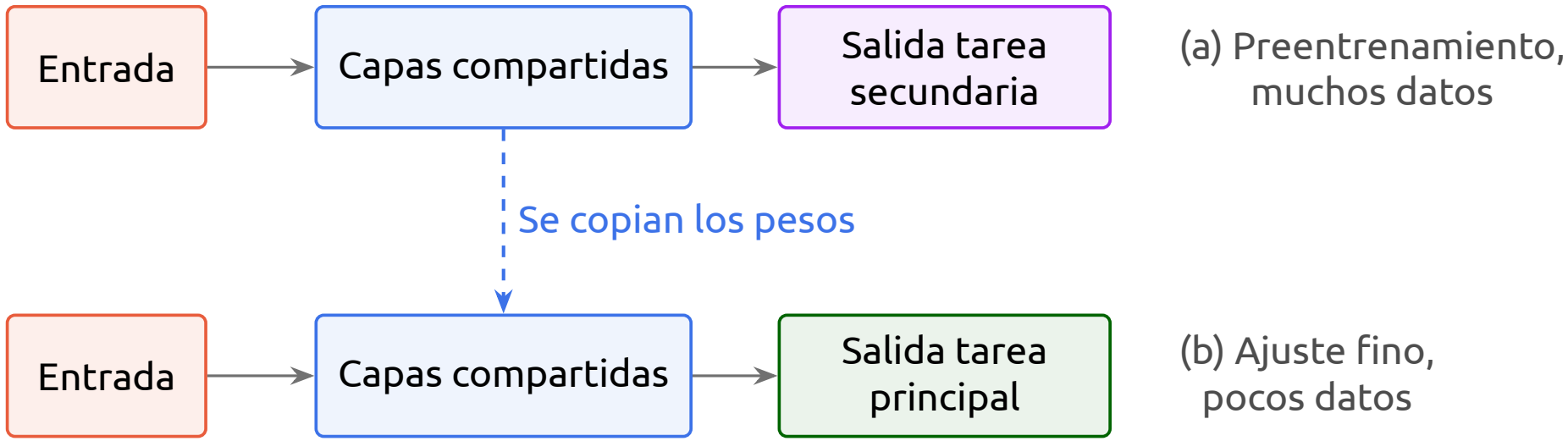
Diez muestras independientes de 15 observaciones, error cuadrático medio (ECM) contra la media verdadera, con los hiperparámetros de las figuras anteriores sin ajustarlos. La penalización L2 y la parada temprana mejoran en siete y ocho de las diez muestras. Dropout, ruido en la entrada y bagging, con estos valores, no mejoran en promedio.

Aumento de datos



Se agregan copias transformadas de los datos con transformaciones que no cambian la etiqueta, aquí desplazamientos de un píxel. Sobre imágenes centradas como las de entrenamiento (a) el aumento no ayuda. Sobre imágenes desplazadas (b) la red sin aumento falla en el 60 % de los casos y con aumento en el 4 %.

Aprendizaje por transferencia

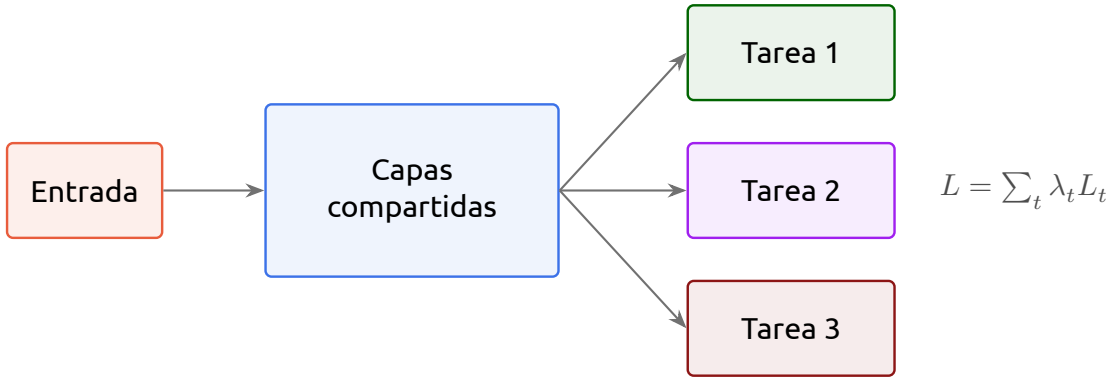


Cuando la tarea principal tiene pocos datos, se entrena primero una red en una tarea relacionada con muchos datos, se reemplazan sus últimas capas por otras adecuadas a la tarea principal y se entrena sobre esta. Las capas copiadas se mantienen fijas o se ajustan con una tasa pequeña.

Aprendizaje multitarea y autosupervisado

Multitarea. Una red con capas compartidas y una salida por tarea se entrena con la suma ponderada de las pérdidas. Cada tarea regulariza a las otras, porque las capas compartidas tienen que servir a todas.

Autosupervisado. Se generan etiquetas a partir de los mismos datos, ocultando una parte de cada ejemplo y pidiendo a la red que la reconstruya, o comparando dos versiones del mismo ejemplo con ejemplos distintos. La red así entrenada se transfiere a la tarea principal.



Inferencia bayesiana

La máxima verosimilitud elige un único $\hat{\phi}$, aunque muchos otros sean casi igual de compatibles con los datos. La inferencia bayesiana asigna a cada ϕ una probabilidad a posteriori,

$$\Pr(\phi \mid \{\mathrm{x}_i, \mathrm{y}_i\}) \propto \Pr(\phi) \prod_{i=1}^{n} \Pr(\mathrm{y}_i \mid \mathrm{x}_i, \phi),$$

y predice promediando sobre todos,

$$\Pr(\mathrm{y} \mid \mathrm{x}, \{\mathrm{x}_i, \mathrm{y}_i\}) = \int \Pr(\mathrm{y} \mid \mathrm{x}, \phi)\ \Pr(\phi \mid \{\mathrm{x}_i, \mathrm{y}_i\})\, d\phi.$$

Es un ensamble infinito ponderado por la plausibilidad de cada parámetro. Con millones de parámetros la integral no es calculable y se aproxima, con ensambles, con dropout de Monte Carlo o con métodos variacionales. Un proceso gaussiano es el límite de una red bayesiana de ancho infinito (Neal, 1996).

Resumen de los métodos de regularización

Método	Mecanismo	Hiperparámetro	En PyTorch
Penalización L2	Pesos pequeños, función suave	λ	`weight_decay` en `AdamW`
SGD, lotes pequeños	Regularización implícita	α, $\lvert\mathcal{B}\rvert$	`lr`, `batch_size`
Parada temprana	Limita el recorrido de los parámetros	Número de pasos	Guardar la mejor época
Ensambles	Promedia modelos, reduce la varianza	Número de modelos	Varias redes
Dropout	Ensamble implícito, redundancia	p	`nn.Dropout(p)`
Ruido en las entradas	Penaliza la derivada de la salida	σ_x	Sumar ruido al lote
Suavizado de etiquetas	Evita salidas extremas	γ	`label_smoothing`
Aumento de datos	Impone invarianzas	Transformaciones	`torchvision.transforms`
Transferencia	A priori aprendido de otros datos	Capas a ajustar	Modelos preentrenados

Todos actúan sobre el término de varianza de la descomposición, a costa de algo de sesgo. Se combinan, y su intensidad se elige por validación.

Síntesis

- Una red ReLU es una función lineal por tramos. Con una capa oculta aproxima cualquier función continua, y la profundidad multiplica el número de regiones lineales por pliegues sucesivos de la entrada.
- La red es una regresión sobre una base que se aprende de los datos. Sus parámetros no son identificables y no se interpretan individualmente.
- La pérdida sale de elegir una distribución para la respuesta. Mínimos cuadrados, entropía cruzada y la pérdida heterocedástica son log verosimilitudes negativas.
- El entrenamiento es descenso de gradiente estocástico o una de sus variantes. La tasa de aprendizaje es el hiperparámetro decisivo.
- La retropropagación calcula el gradiente completo con un costo de unos pocos pases hacia adelante, y la inicialización de He conserva la escala de activaciones y gradientes.
- Las redes sobreparametrizadas pueden generalizar después de memorizar, y la regularización, explícita o implícita, se verifica con datos de validación.

Errores frecuentes

- Aplicar la softmax antes de `nn.CrossEntropyLoss` o la sigmoide antes de `nn.BCEWithLogitsLoss`. Ambas pérdidas reciben las salidas sin transformar.
- Olvidar `optimizador.zero_grad()`, con lo que los gradientes se acumulan entre iteraciones.
- Evaluar sin `modelo.eval()`, con el dropout activo.
- No estandarizar las entradas, lo que empeora el condicionamiento de la pérdida.
- Elegir la arquitectura o la época de parada mirando el conjunto de test.
- Reportar sólo el error de clasificación cuando se usarán las probabilidades predichas, sin revisar la pérdida de validación ni la calibración.
- Interpretar el valor de un peso individual como si fuera un coeficiente de regresión.
- Concluir que un regularizador funciona a partir de una sola corrida, sin repetir con otras semillas o muestras.

Siglas

Sigla	Significado
ECM	Error cuadrático medio
GELU	Gaussian error linear unit
GPU	Unidad de procesamiento gráfico (graphics processing unit)
KL	Kullback–Leibler
MLP	Perceptrón multicapa (multilayer perceptron)
ReLU	Unidad lineal rectificada (rectified linear unit)
SGD	Descenso de gradiente estocástico (stochastic gradient descent)
SiLU	Sigmoid linear unit

Referencias

- Prince, S. J. D. (2023). *Understanding Deep Learning*, capítulos 3 a 9. MIT Press.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*, capítulos 6 a 8. MIT Press.
- Bishop, C. M., Bishop, H. (2024). *Deep Learning: Foundations and Concepts*, capítulos 6 a 9. Springer.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.
- Montúfar, G., Pascanu, R., Cho, K., Bengio, Y. (2014). On the number of linear regions of deep neural networks. *NeurIPS*.
- Kingma, D. P., Ba, J. (2015). Adam: a method for stochastic optimization. *ICLR*.
- He, K., Zhang, X., Ren, S., Sun, J. (2015). Delving deep into rectifiers. *ICCV*.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. (2014). Dropout. *Journal of Machine Learning Research*, 15, 1929–1958.
- Nix, D. A., Weigend, A. S. (1994). Estimating the mean and variance of the target probability distribution. *IEEE ICNN*.
- Belkin, M., Hsu, D., Ma, S., Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *PNAS*, 116(32), 15849–15854.
- Barrett, D. G. T., Dherin, B. (2021). Implicit gradient regularization. *ICLR*.